ASIGNATURA:
APLICACIONES DE MACHINE LEARNING

SISTEMAS INTELIGENTES



2. Aplicaciones de ML en Clasificación

2.1 Revisión de modelos y métricas

2.2 Randomized Search

2.3 Optimización Bayesiana

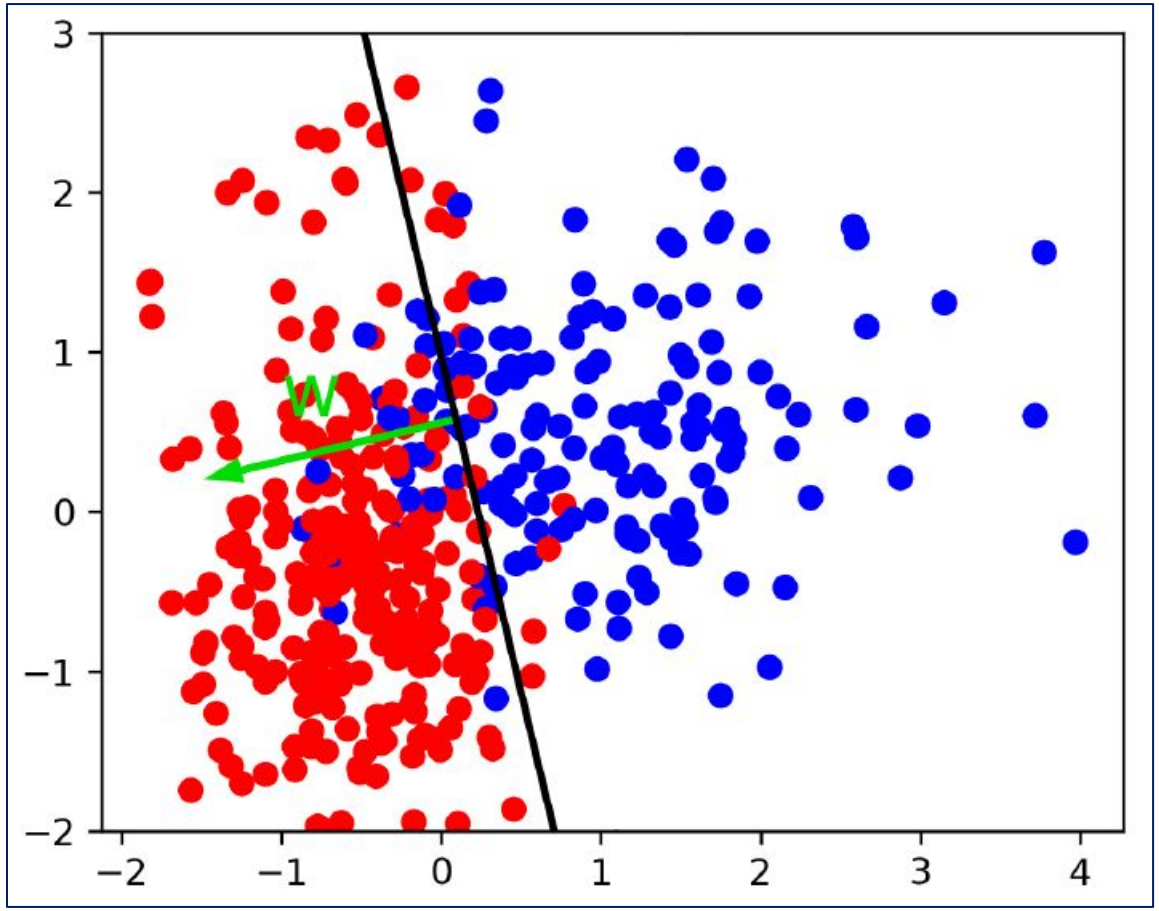
Regresión Logística

Método supervisado utilizado cuando la variable a predecir **"y"** es categórica, normalmente **binaria,** ejemplo:

Ejemplos:

¿El cliente comprará? (Sí/No)

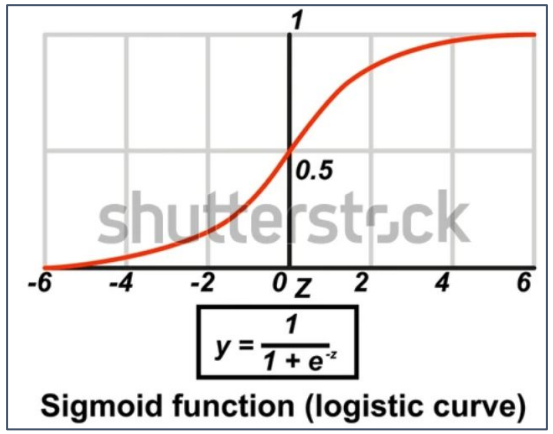
¿El correo es spam? (Sí/No).



Partimos de la regresión lineal:

$$z = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n$$

Función logística (sigmoid): Para transformar z en probabilidad:



$$p(y = 1|x) = \frac{1}{1 + e^{-z}}$$

Esta función transforma cualquier número real en un valor entre 0 y 1, y por lo tanto probabilidad.

Si $p \geq 0.5 \rightarrow$ clase 1
Si $p < 0.5 \rightarrow$ clase 0

$$P(está\ enfermo) = \frac{1}{1 + e^{-(b_1 \cdot edad + b_2 \cdot sexo + b_3 \cdot hábito_de_fumar + a)}}$$

Debemos encontrar los pesos y el intercepto

Función de pérdida:

$$J(\mathbf{w}) = \frac{1}{2n} \sum_{i=1}^{n} \log(\exp(-y_i(w^T \mathbf{x}_i + b)) + 1)$$

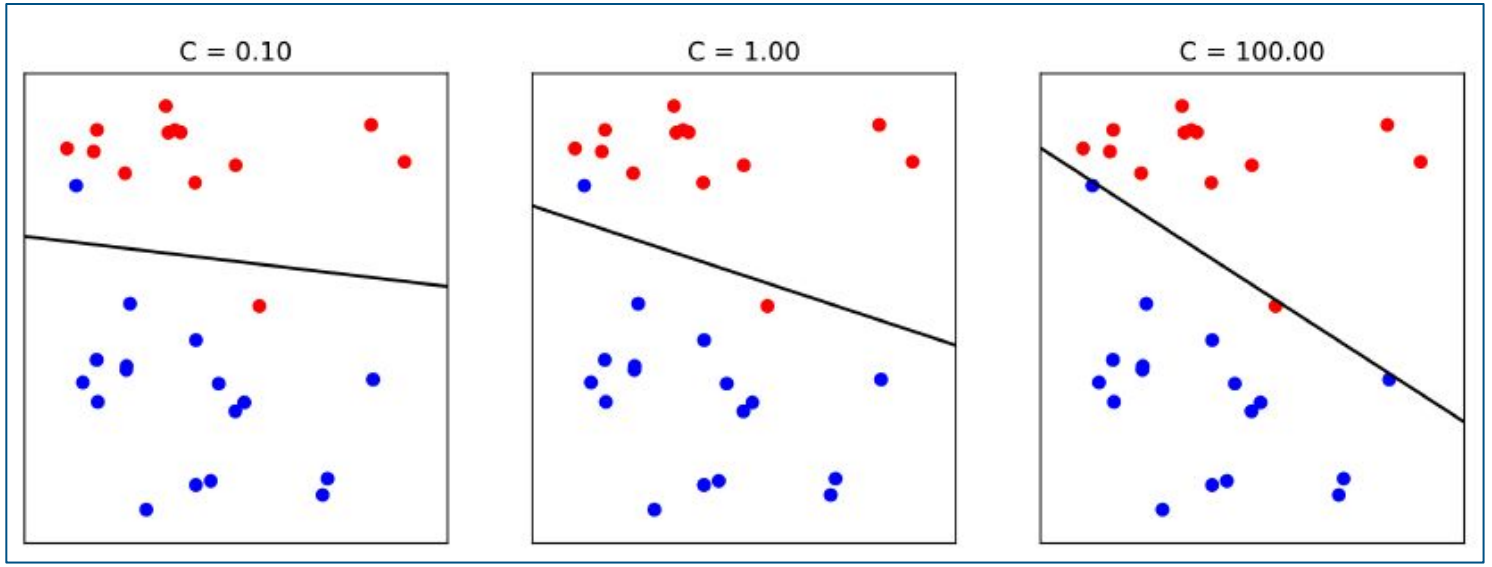
- $\min_{w \in P, b \in \mathbb{R}} C \sum_{i=1}^{n} \log(\exp(-y_i(w^T \mathbf{x}_i + b)) + 1) + ||w||_2^2$ ~Ridge L2
- $\min_{w \in P, b \in \mathbb{R}} C \sum_{i=1}^{n} \log(\exp(-y_i(w^T \mathbf{x}_i + b)) + 1) + ||w||_1$ ~Lasso L1

Loss 1 Loss 2

C parámetro de regularización inversa
C~1/α; C tiene efecto inverso al parámetro de regularización α.
Loss1 mide qué tan bien el modelo clasifica los datos (error logístico).
Loss2 penaliza pesos grandes, reduciendo la complejidad del modelo (sobreajuste).

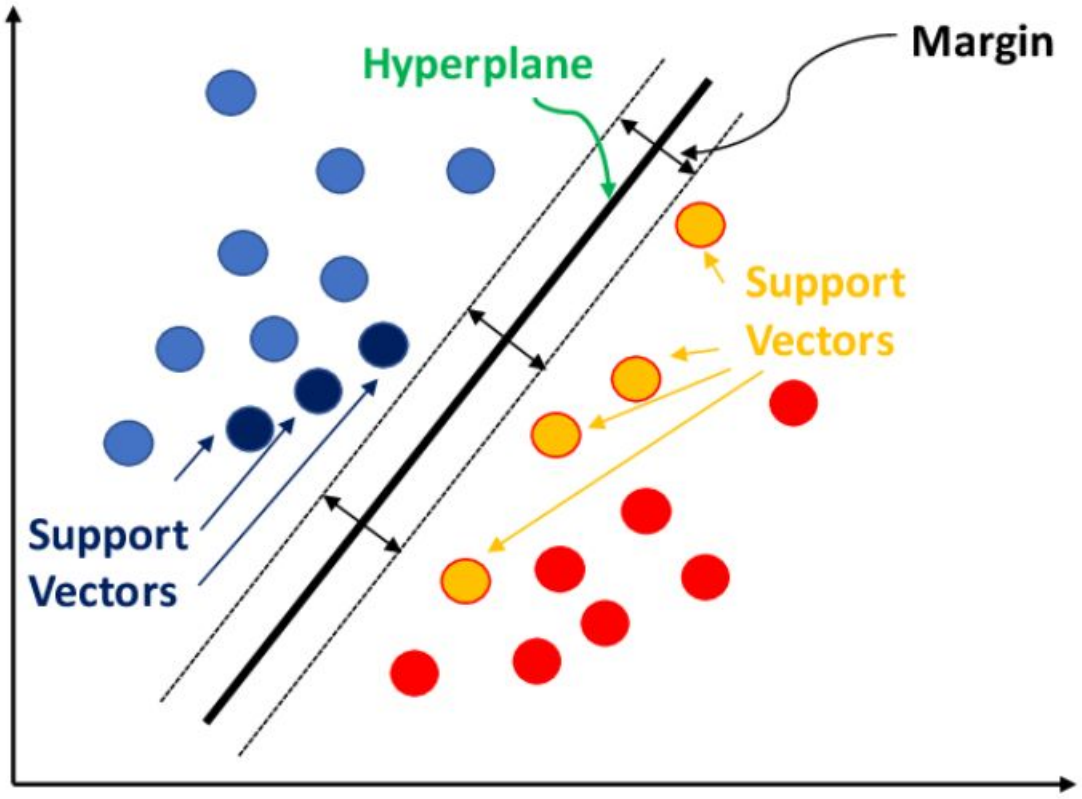
“C” o fuerza inversa de regularización:

Figura	Valor de C	Interpretación
Izquierda	$C = 0.10$	Mucha regularización → los coeficientes son pequeños → la frontera de decisión es más “suave” (algunos errores son tolerados).
Centro	$C = 1.00$	Regularización moderada → balance entre ajuste y simplicidad.
Derecha	$C = 100.00$	Poca regularización → el modelo trata de separar perfectamente los puntos → riesgo de sobreajuste.



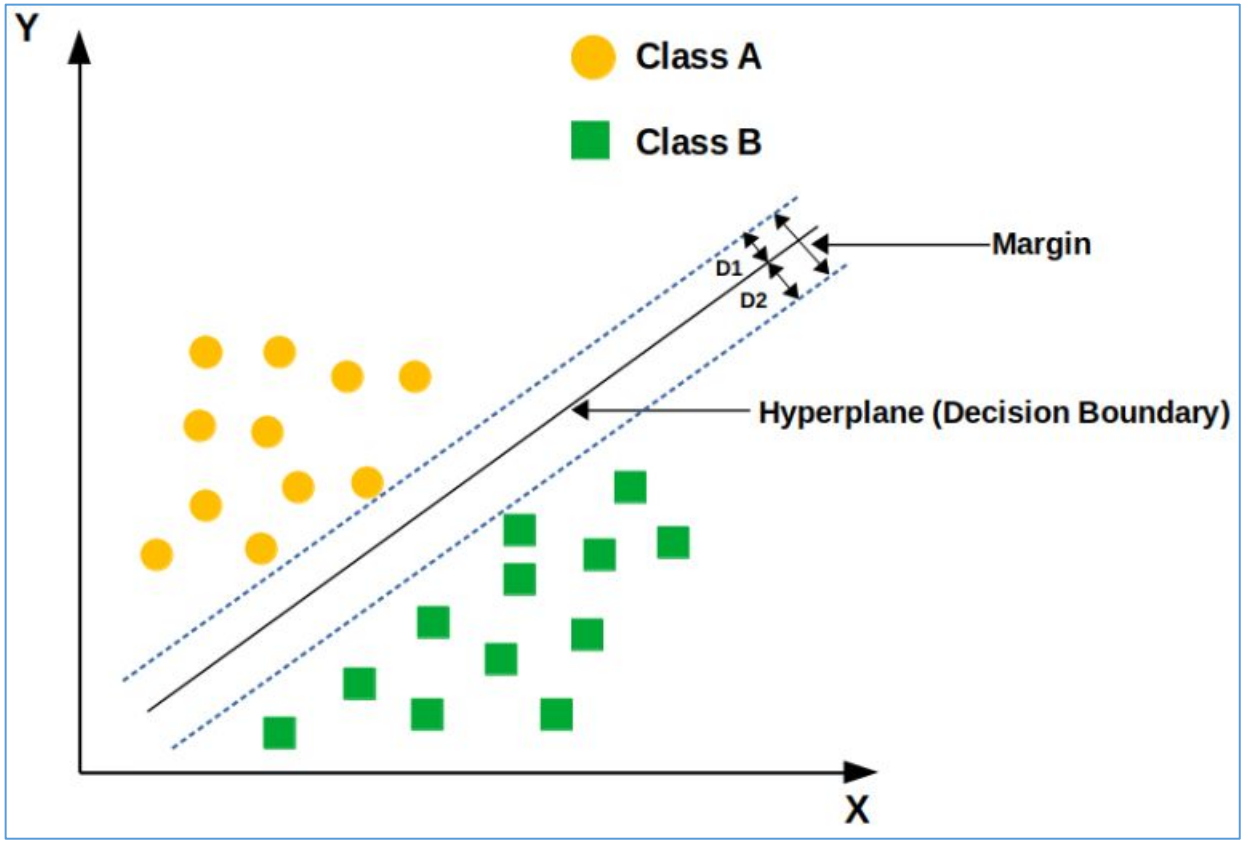
Support Vector Classifier (SVC)

Imagine 2 clases (azul-rojo) como puntos en un plano.
SVM busca la línea que separa las clases con el mayor margen posible, dejando el espacio vacío más grande entre ambos.

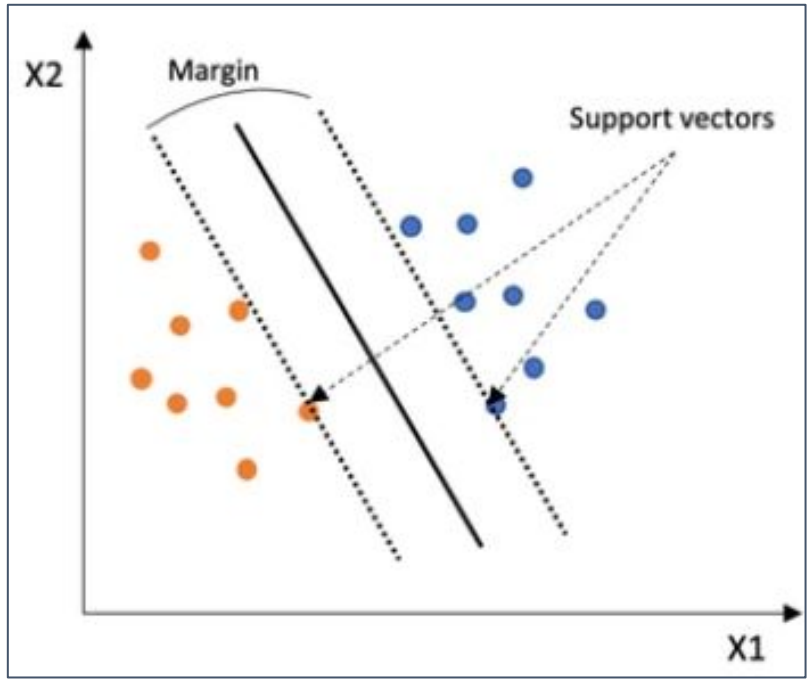


El margen es la distancia entre el hiperplano/recta y los puntos más cercanos de cada clase.
Margen equivale a Tubo.

Un margen más ancho significa que el modelo separa las clases con más confianza y podrá generalizar mejor.



Vectores de Soporte:
Son los puntos de datos más críticos, los que están más cerca del límite de decisión.
Ellos definen la posición y orientación del hiperplano



Hiperplano de decisión

$w^T x+b=0$

Divide el espacio en dos clases:

- $>0 \rightarrow$ clase 1
- $<0 \rightarrow$ clase 0

Cuando no se puede separar perfecto:

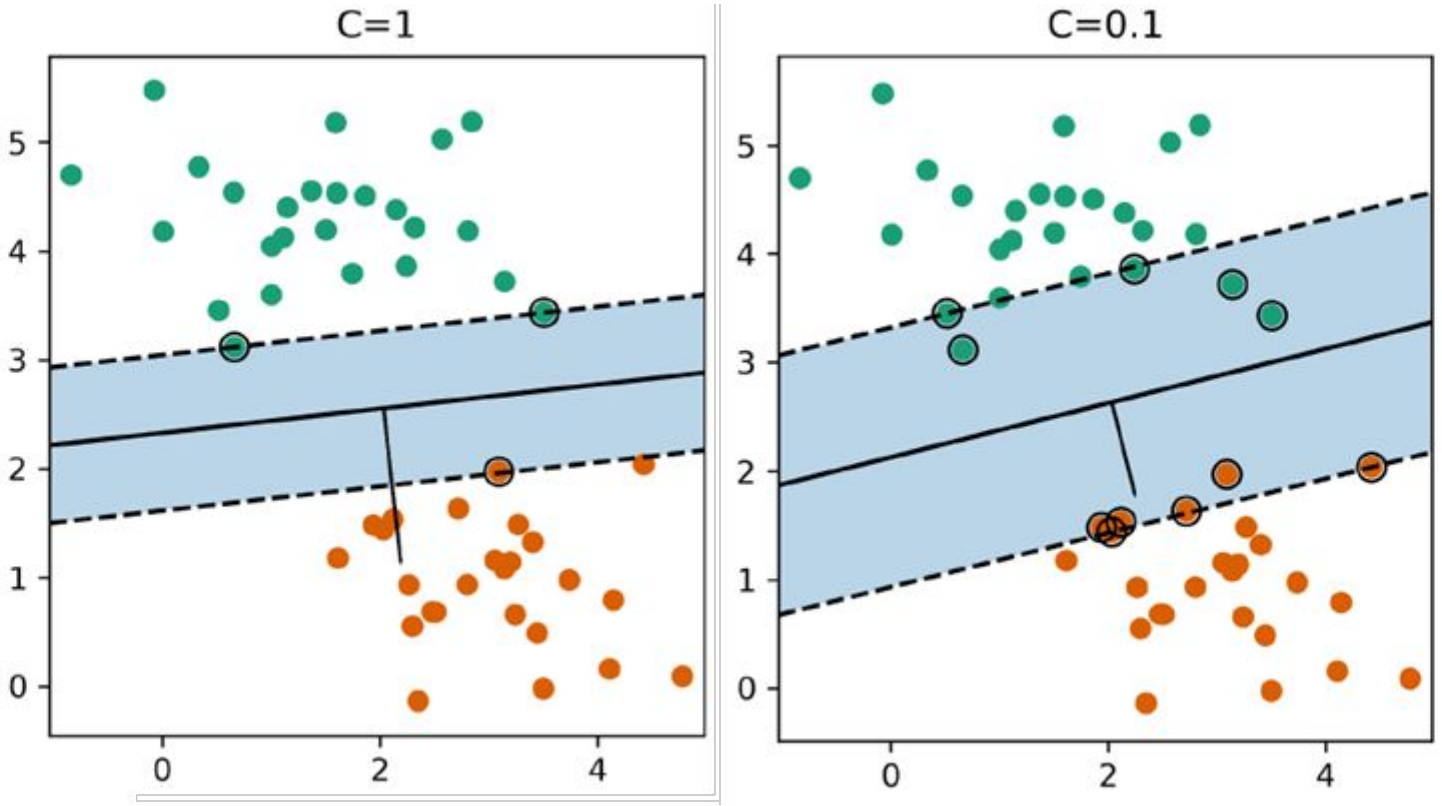
$$\min \frac{1}{2}||w||^2 + C\sum \xi_i$$

- ξ_i = errores
- C = penalización

'C' es un **parámetro de penalización** que controla **cuánto tolera el modelo los errores de clasificación** en los datos de entrenamiento. C multiplica el error.

$$FO = \frac{1}{2}||w||^2 + C\sum \xi_i$$

C grande → Tolera menos errores.
C pequeño → Tolera más errores.



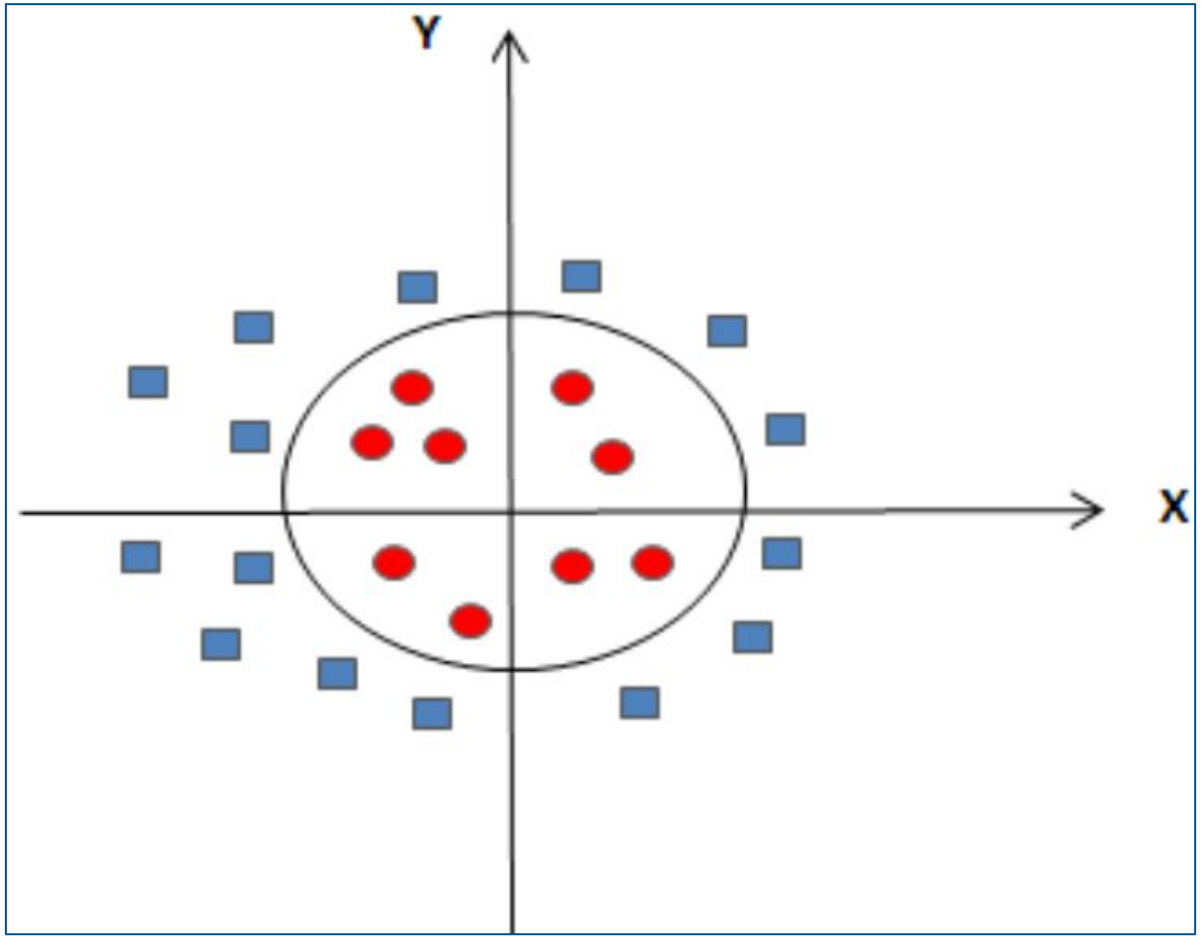
Valor de C	Comportamiento del modelo	Resultado típico
C Grande	Penaliza mucho los errores → **Margen más estrecho** → Menos puntos mal clasificados en entrenamiento	Menos errores en entrenamiento. Riesgo de **sobreajuste (overfitting)**.
C Pequeño	Penaliza poco los errores (**tolera algunos errores**) → **Margen más amplio**→ Más errores en entrenamiento. Modelo más simple.	Mayor generalización. Riesgo de **subajuste (underfitting).**

¿Cómo tratar datos no linealmente separables?

Imaginemos un caso en el que no existe una línea recta (o hiperplano) que pueda separar dos clases.

En la imagen que se muestra a continuación, hay un círculo en 2D con puntos de datos rojos y azules distribuidos por toda su superficie.

SVM aborda este caso mediante el uso de una función kernel para manejar datos no linealmente separables.

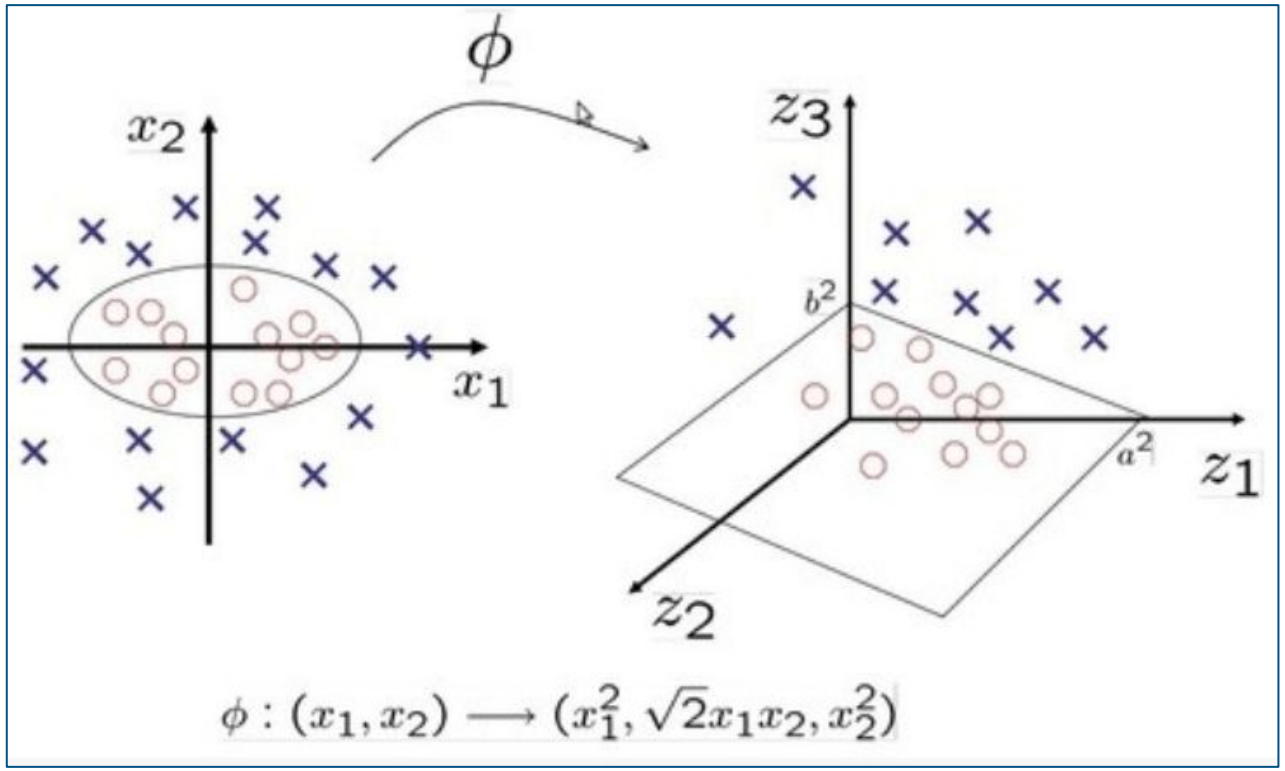


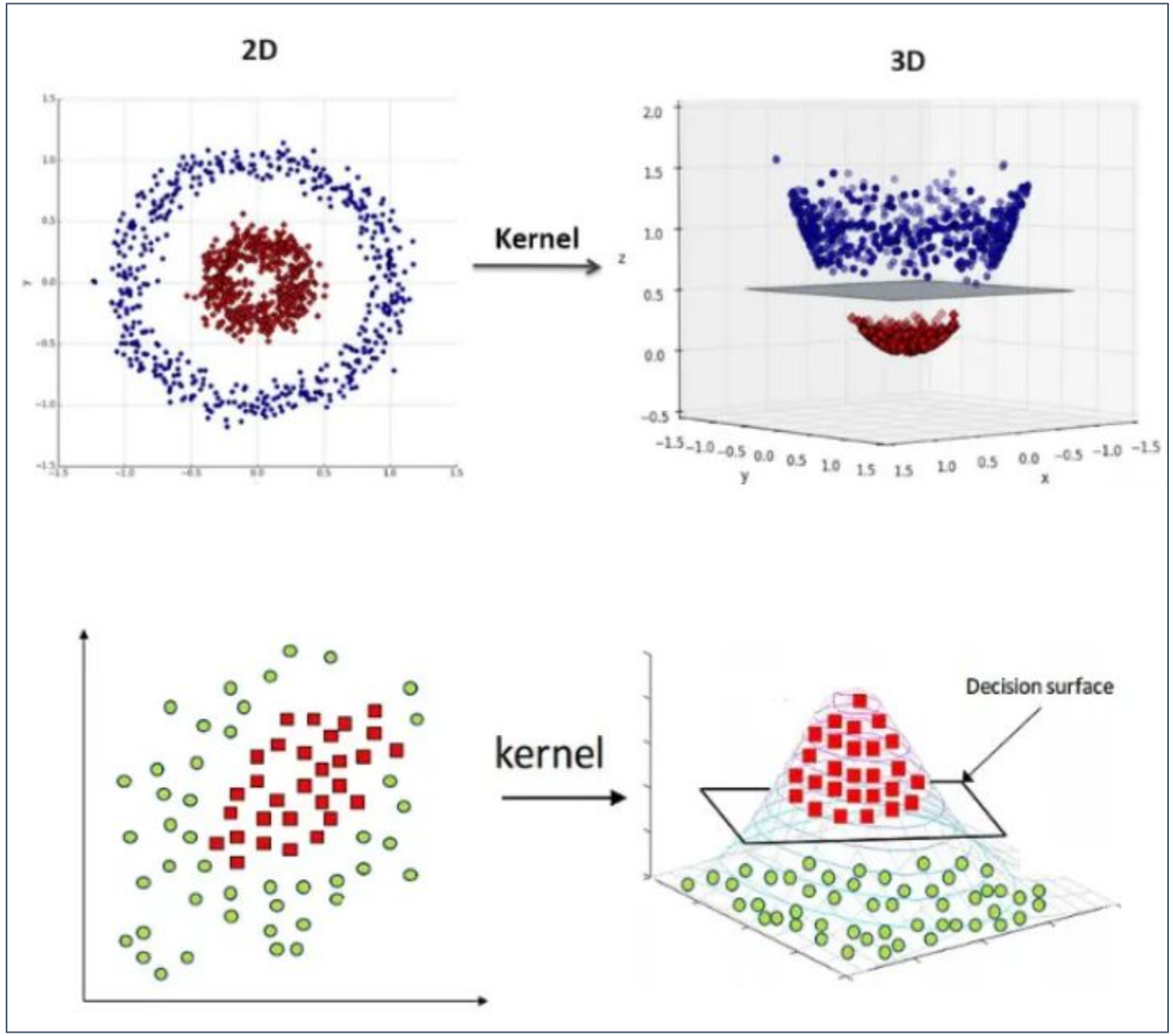
¿Que es un Kernel?

Kernel es un método para que SVM funcione con datos no linealmente separables.

La función kernel **transforma** los datos a un espacio de características de **mayor dimensión** para permitir la separación lineal.

Algunos funciones kernel son:
-Linear
-Polynomial kernel
-Radial basis function kernel (RBF kernel) *
-Sigmoid kernel



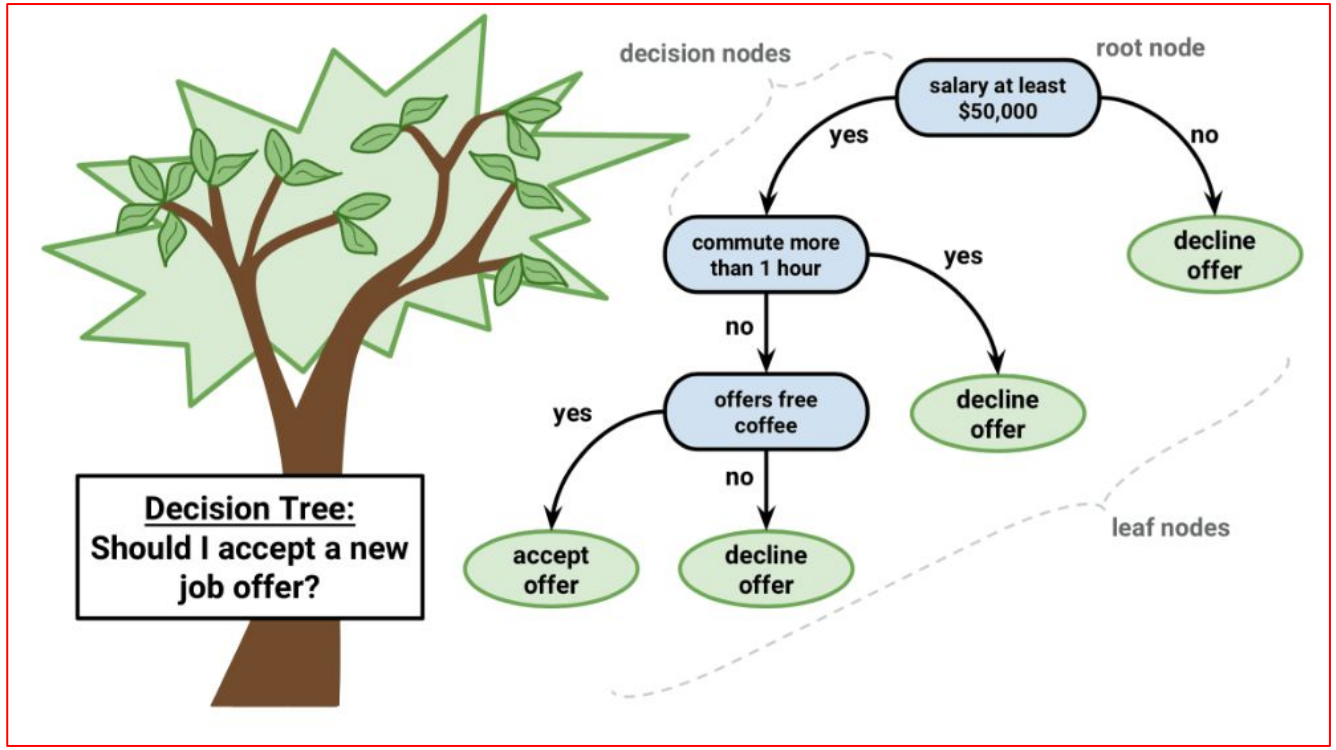

2D
3D
Kernel
kernel
Decision surface

Árboles para Clasificación

Modelo supervisado utilizado para **clasificación o regresión**, basada en particiones/decisiones.

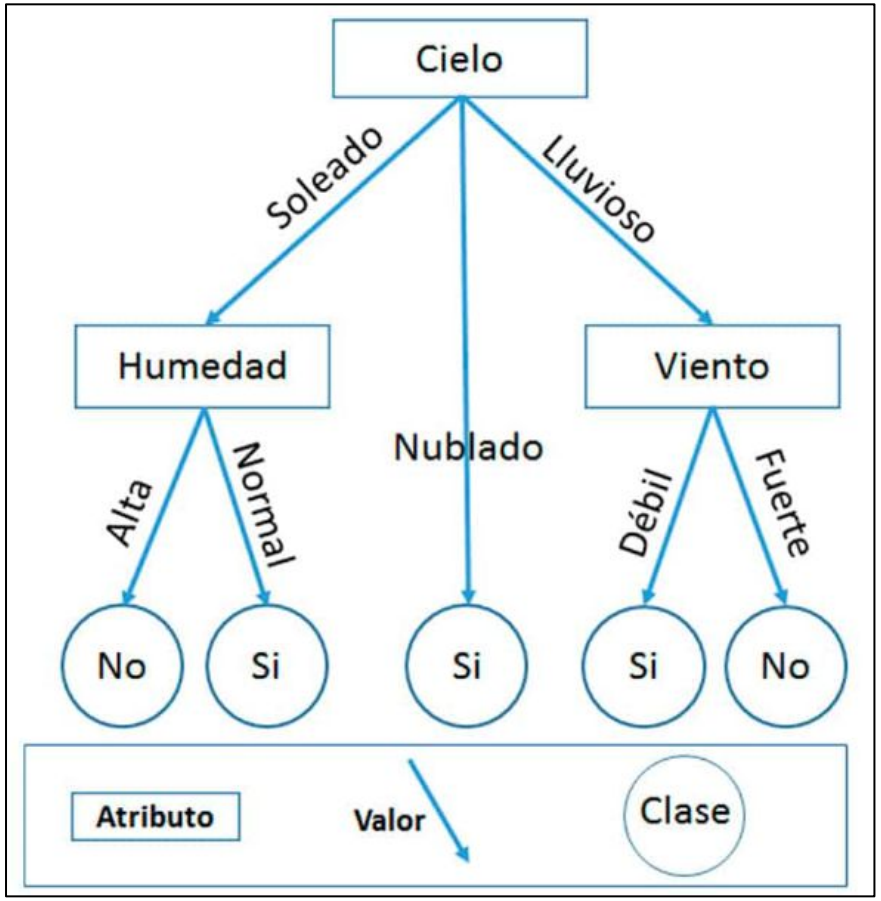
Divide el dataset en partes pequeñas, eligiendo la opción mas informativa en cada paso.

Se divide la población o muestra en conjuntos homegéneos basados en la variable de entrada con mas información.



Dataset: ¿Jugar tenis?

Cielo	Temperatura	Humedad	Viento	Jugar
soleado	caliente	alta	falso	no
soleado	caliente	alta	verdadero	no
nublado	caliente	alta	falso	sí
lluvioso	templado	alta	falso	sí
lluvioso	fresco	normal	falso	sí
lluvioso	fresco	normal	verdadero	no
nublado	fresco	normal	verdadero	sí
soleado	templado	alta	falso	no
soleado	fresco	normal	falso	sí
lluvioso	templado	normal	falso	sí
soleado	templado	normal	verdadero	sí
nublado	templado	alta	verdadero	sí
nublado	caliente	normal	falso	sí
lluvioso	templado	alta	verdadero	no



Se representa como un árbol con:
Nodo- Pregunta o condición de una variable/feature/atributo.
Rama.- Es una respuesta o decisión.
Hoja (leaf).- Es una predicción/resultado final/clase.

Entropia

Entropia H(S) es una medida de la **cantidad de incertidumbre** en el data set S.

$$H(S) = \sum_{c \in C} -p(c) log_2 p(c)$$

C es el conjunto de clases en S, por ejemplo, C={yes,no}.
p(c) La proporción del número de elementos de la clase C con respecto al número de elementos del conjunto S.

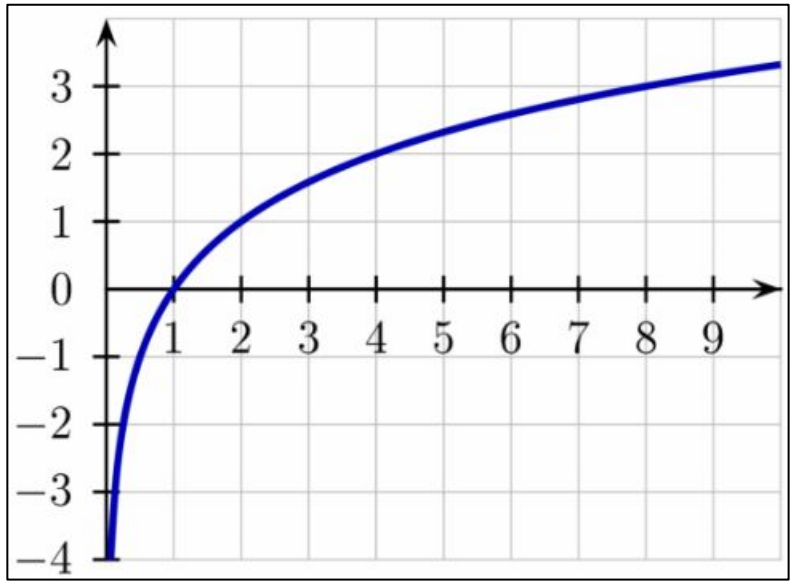
Si H(s)=1, es la máxima entropía, es decir, **máxima incertidumbre**.
Si H(s)=0, es la mínima entropía, y el subconjunto S esta perfectamente clasificado, máxima pureza, **máxima certeza.**

Evento: cara o sello.
Calculo de Entropía:

E=-1*[1/2*log(1/2) + 1/2*log(1/2)]
E=-1*[1/2*(-1) + ½*(-1)]
E= ½+½ =1

Evento: moneda de 2 caras iguales

E=-1*[2/2*log(2/2) + 0/2*log(0/2)]
E=-1*[1*0 + 0]
E= 0



Ganancia de Información

IG(A) es la medida de la diferencia en entropía “antes” y ”después” de dividir el dataset S basado en un **atributo A.**
En otras palabras, cuanta incertidumbre se reduce luego de dividir el data set en el **atributo A.**

$$IG(A, S) = \mathbf{H}(S) - \sum_{t \in T} p(t) H(t)$$

H(s) es la entropía del conjunto S.
T es el subconjunto creado de dividir el conjunto S por el atributo A.
p(t) La proporción del número de elementos en *t* para el numero de elementos en el conjunto *S*.
Si H(t) , es la entropía del subconjunto t.

outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

Que feature elegir como nodo raíz?

Calcular la entropía:

$$H(S) = \sum_{c \in C} -p(c) log_2 p(c)$$

$$C = \{yes, no\}$$

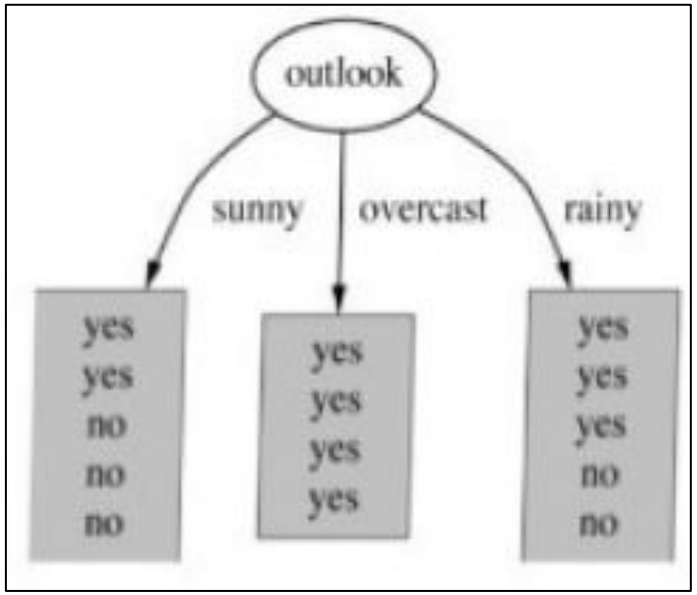
De las 14 observaciones, 9 son ‘yes’, 5 son ‘no’.

$$p(yes) = -(9/14) * log_2(9/14)$$

$$p(no) = -(5/14) * log_2(5/14)$$

$$H(S) = p(yes) + p(no) = 0.94$$

outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no



E (Outlook=sunny) = $-\frac{2}{5}\log\left(\frac{2}{5}\right)-\frac{3}{5}\log\left(\frac{3}{5}\right)=0.971$

E (Outlook=overcast) = $-1\log(1)-0\log(0)=0$

E (Outlook=rainy) = $-\frac{3}{5}\log\left(\frac{3}{5}\right)-\frac{2}{5}\log\left(\frac{2}{5}\right)=0.971$

} H(S,Outlook)

Average Entropy information for Outlook

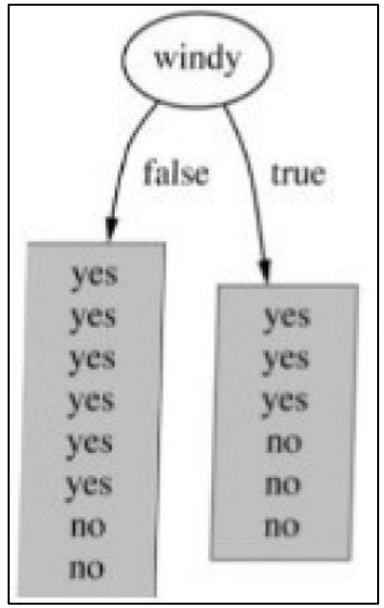
I (Outlook) = $\frac{5}{14}*0.971+\frac{4}{14}*0+\frac{5}{14}*0.971=0.693$

$\sum_{t\in T} p(t)H(t)$

Gain (Outlook) = E(S) – I (outlook) = 0.94-.693 = 0.247 ⇨ $IG(A,S)=H(S)-\sum_{t\in T}p(t)H(t)$

IG(outlook)= 0.247

outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no



$$E\ (\text{Windy=false}) = -\frac{6}{8}\log\left(\frac{6}{8}\right) - \frac{2}{8}\log\left(\frac{2}{8}\right) = 0.811$$

$$E\ (\text{Windy=true}) = -\frac{3}{6}\log\left(\frac{3}{6}\right) - \frac{3}{6}\log\left(\frac{3}{6}\right) = 1$$

Average entropy information for Windy

$$I\ (\text{Windy}) = \frac{8}{14} * 0.811 + \frac{6}{14} * 1 = 0.892$$

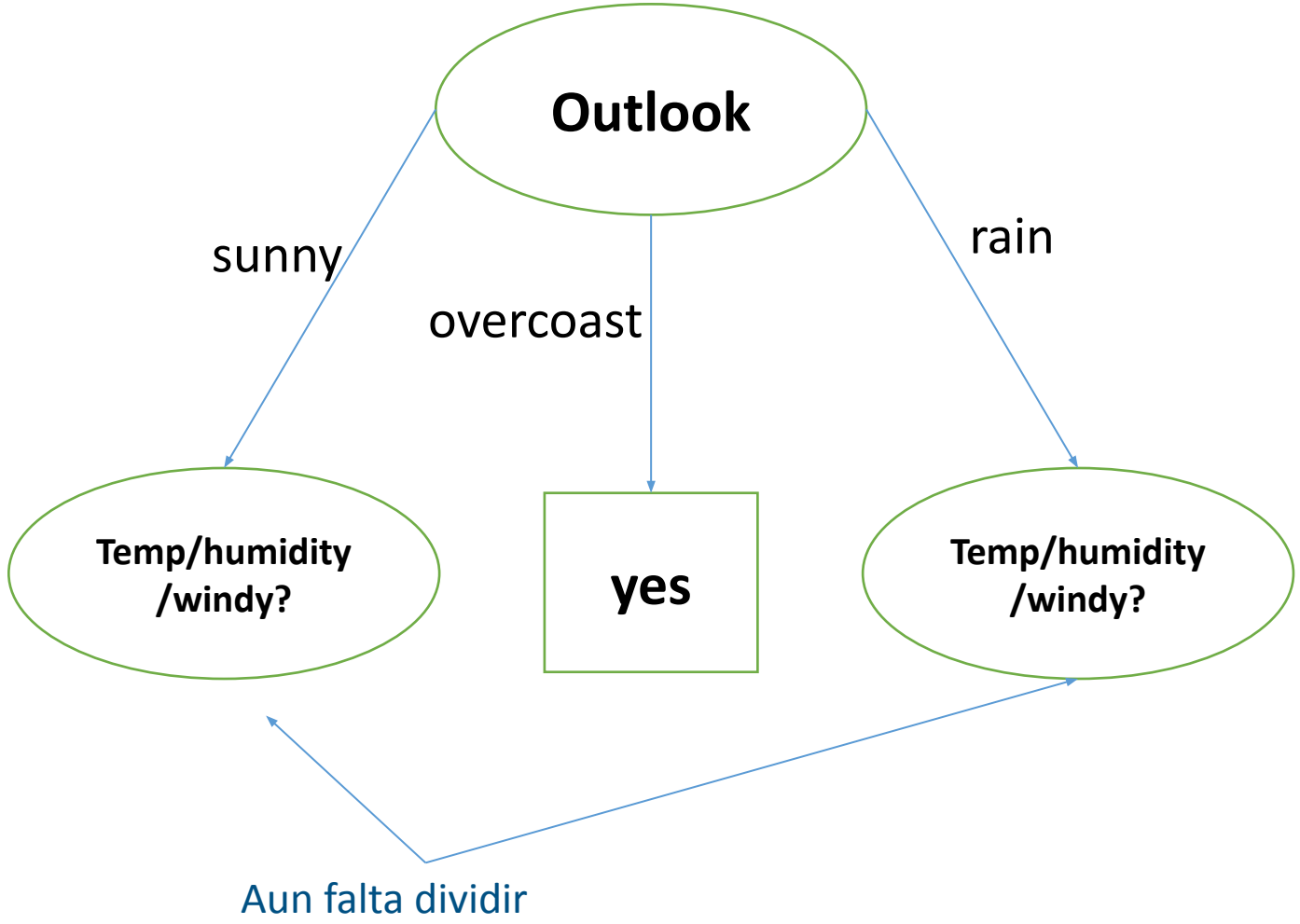
Gain (Windy) = E(S) – *I* (Windy) = 0.94-0.892=0.048

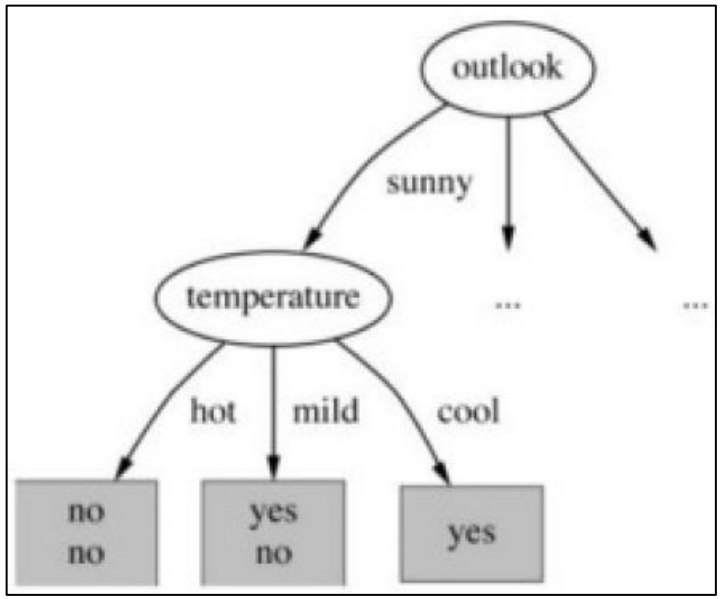
IG(windy)= 0.048

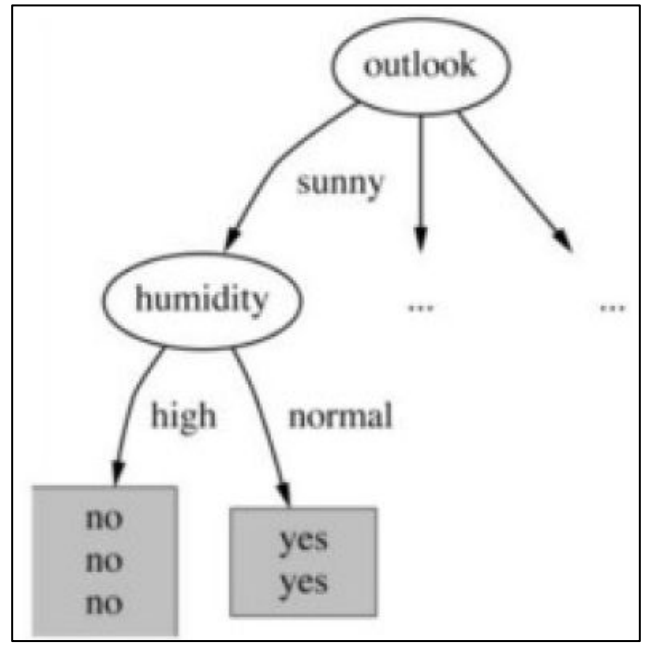
outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

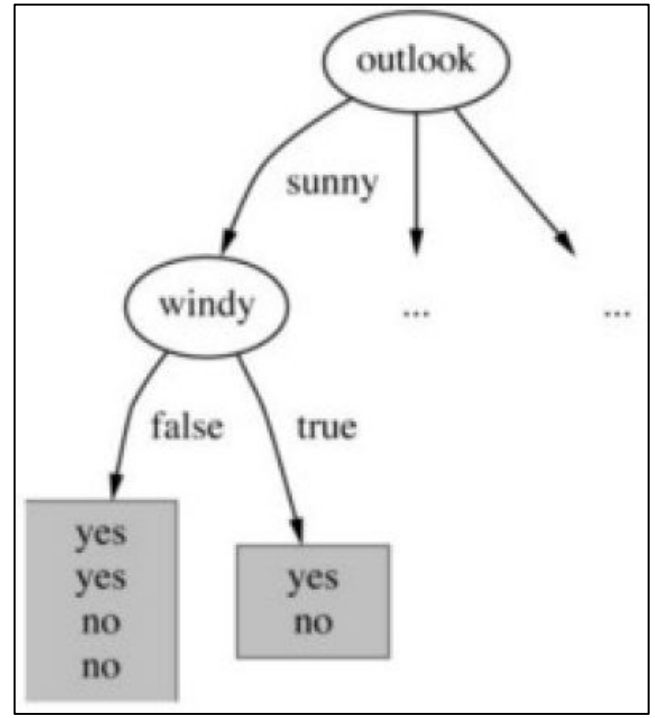
Outlook		Temperature	
Gain: 0.940-0.693	0.247	Gain: 0.940-0.911	0.029
Humidity		**Windy**	
Gain: 0.940-0.788	0.152	Gain: 0.940-0.892	0.048

outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no



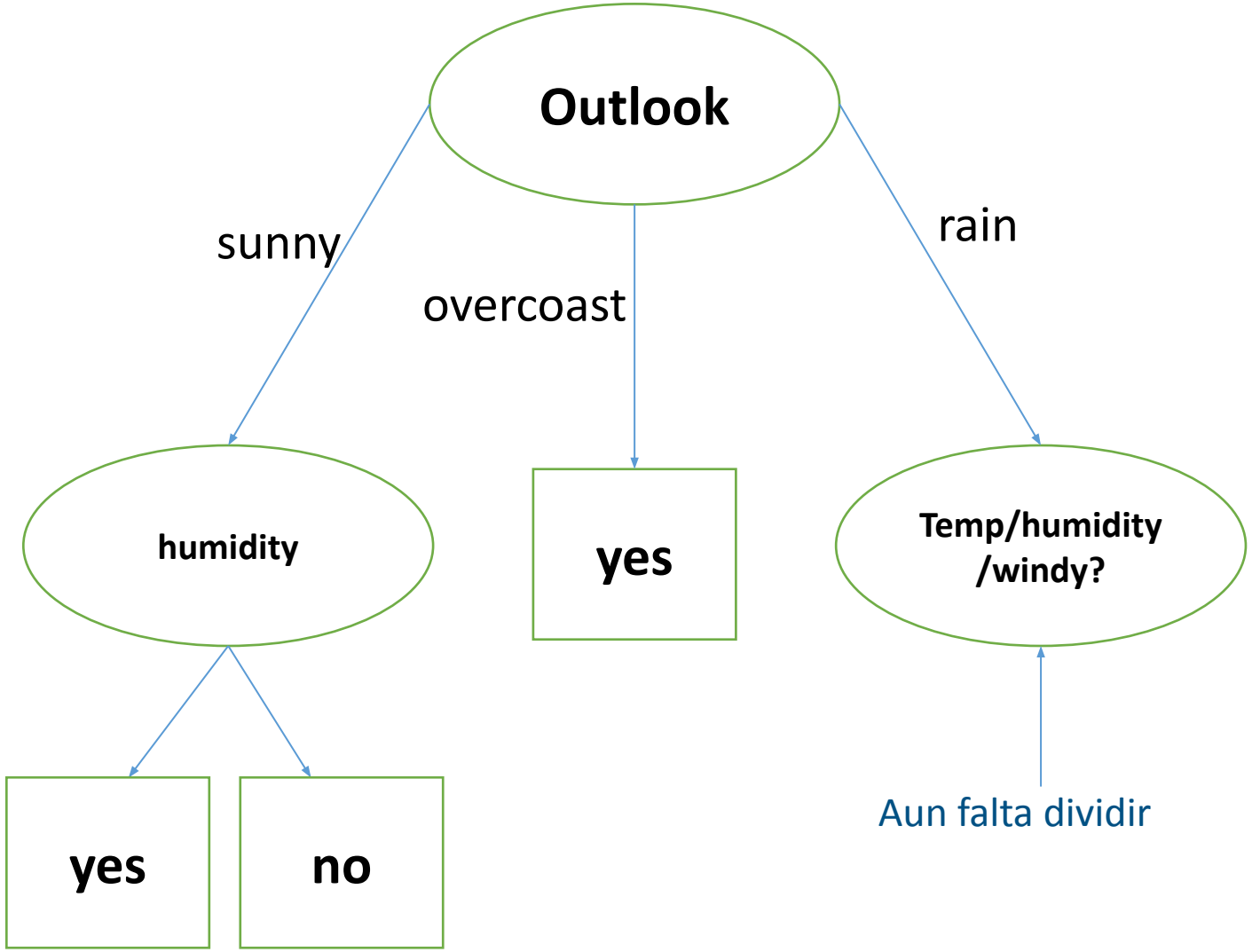






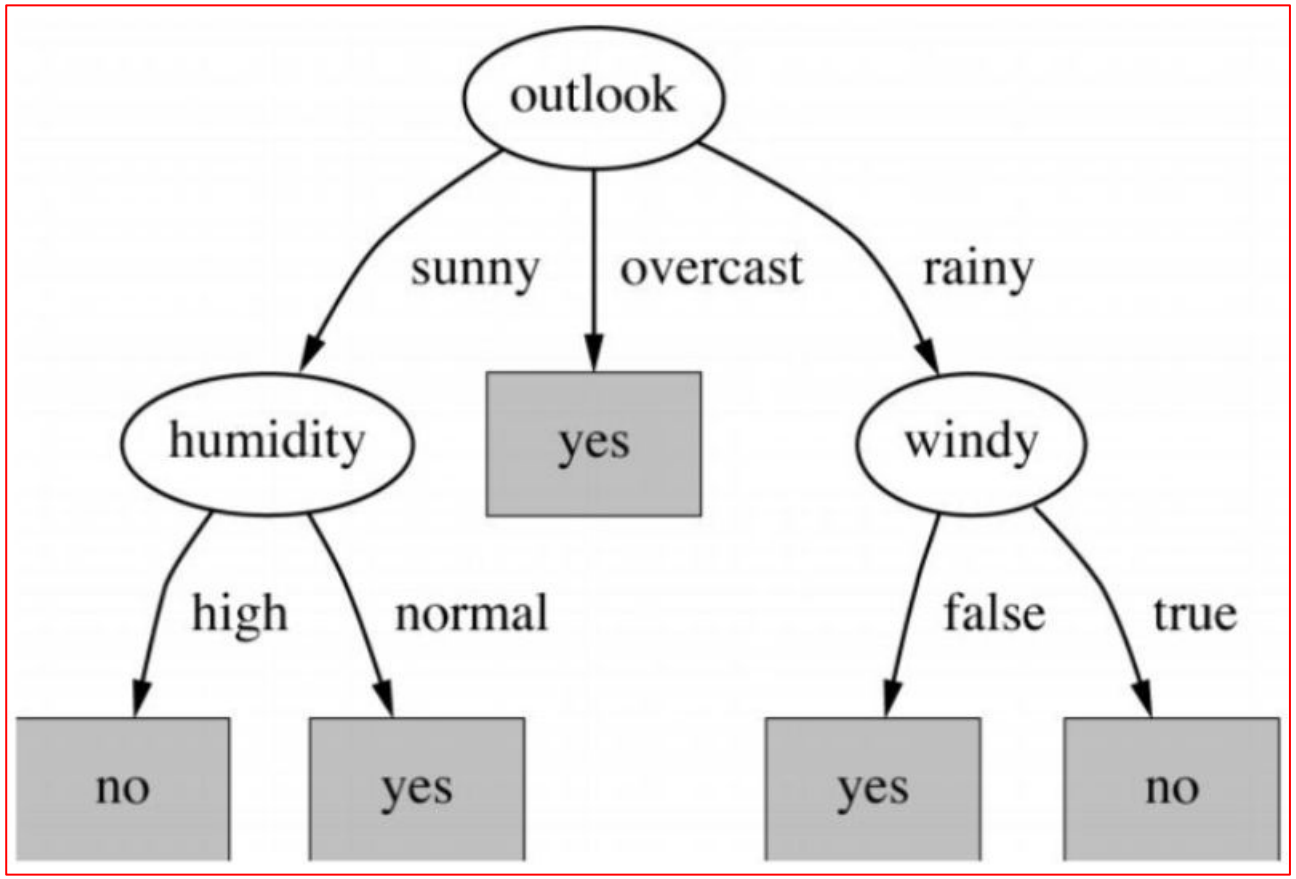
Gain(*Temperature*) = 0.571 bits
Gain(*Humidity*) = 0.971 bits
Gain(*Windy*) = 0.020 bits

outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

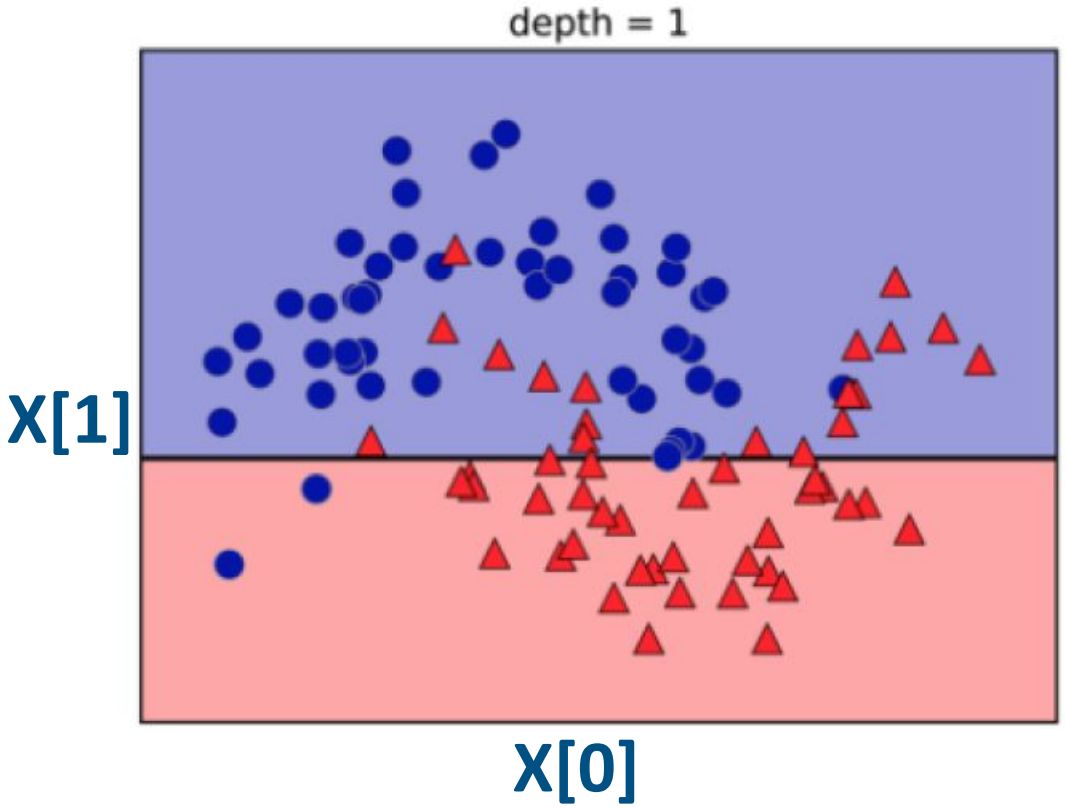


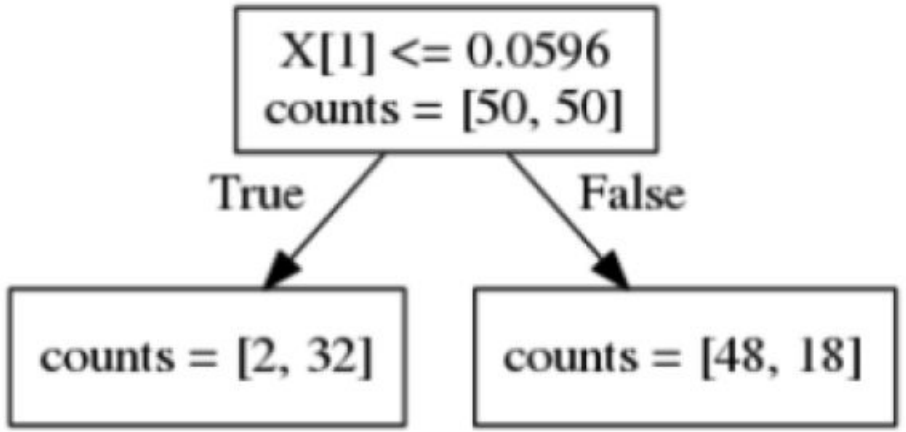
outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

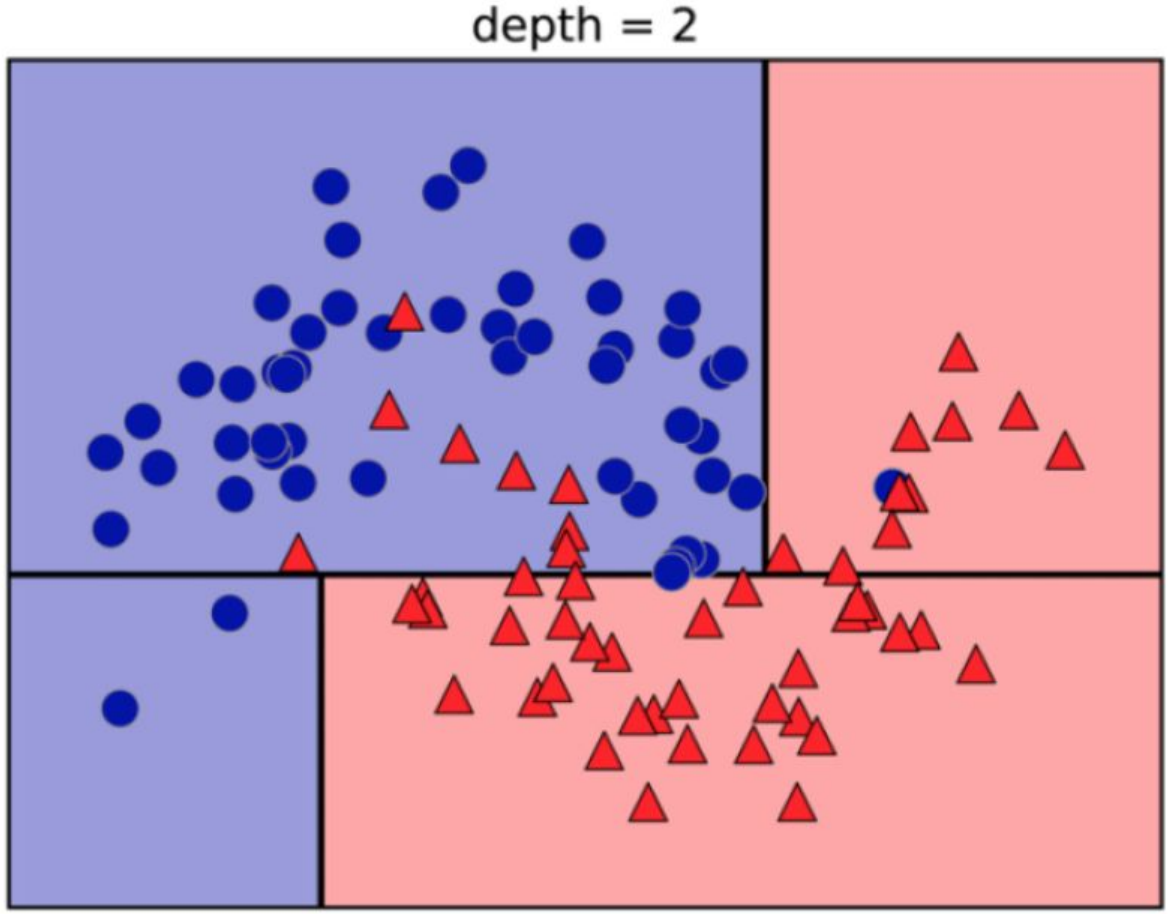
Depth = 2

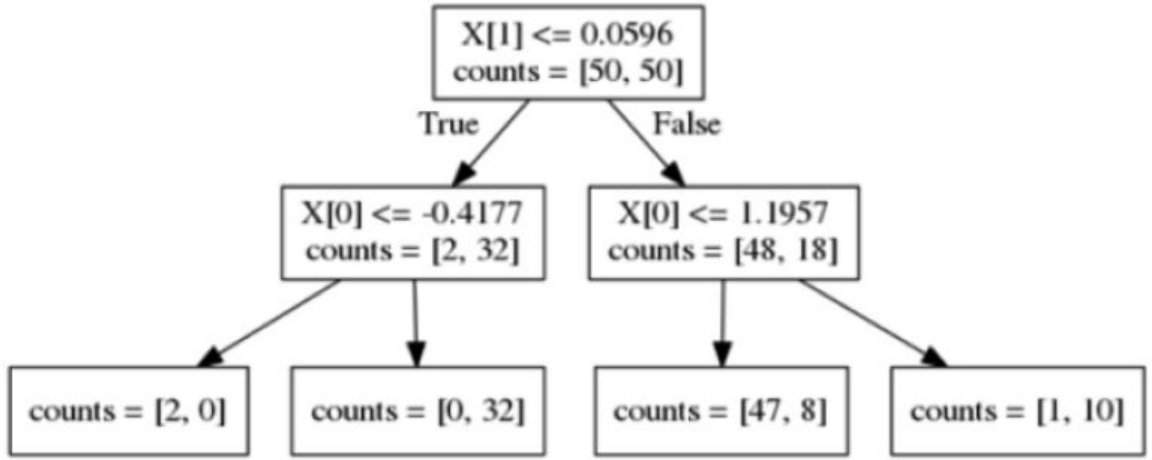


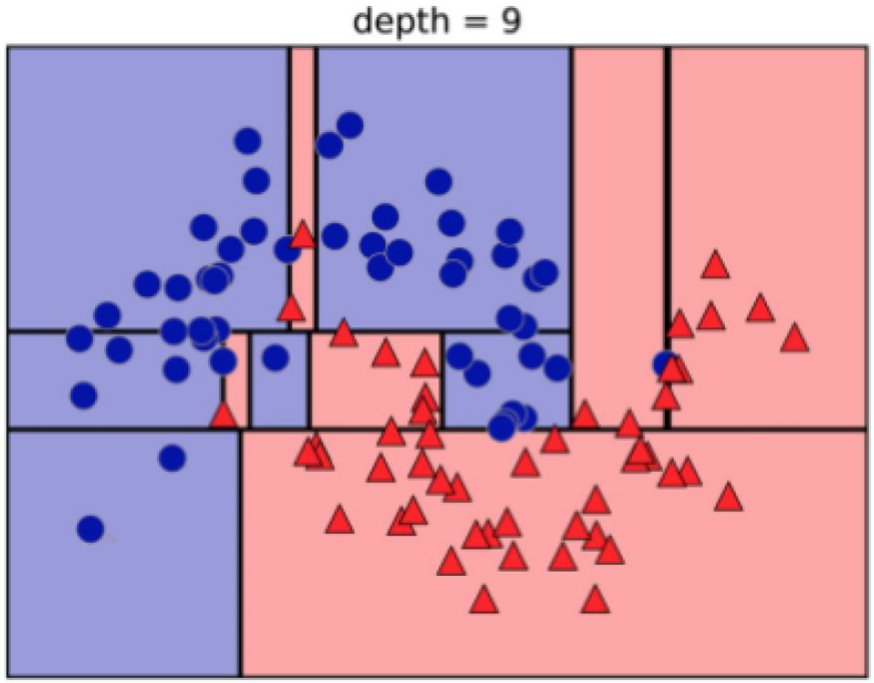
Otro Ejemplo






depth = 2


X[1] <= 0.0596
counts = [50, 50]
True
False
X[0] <= -0.4177
counts = [2, 32]
X[0] <= 1.1957
counts = [48, 18]
counts = [2, 0]
counts = [0, 32]
counts = [47, 8]
counts = [1, 10]



Hay que poner un limite para evitar el sobreajuste:

Max_Depth

Max_leaf_nodes

Min_sample_split

Métricas en Clasificación

True Positive (TP): El caso es positivo en la realidad y el modelo predice Positivo.
True NegatIve (TN): El caso es negativo en la realidad y el modelo predice Negativo.

False Positive (FP): El caso es negativo en la realidad y el modelo predice Positivo.
Error tipo 1. Falsa Alarma.
Paciente sano, pero el test ML le dio resultado Positivo.

False Negative (FN): El caso es positivo en la realidad y el modelo predice Negativo.
Error tipo 2. Fallo de Detección (Grave)

Métricas Clasificación

Definiciones previas:

En un problema de clasificación binaria, **Matriz de confusión**:

		Predice “Sí”	Predice “No”
Enfermo	**Real = Sí**	✅ Verdadero Positivo (TP)	❌ Falso Negativo (FN)
No enfermo (sano)	**Real = No**	❌ Falso Positivo (FP)	✅ Verdadero Negativo (TN)

Ejemplo: Detección de enfermedad.

TP: El paciente está enfermo y modelo predijo que lo está.

TN: El paciente está sano y modelo predijo que lo está.

FN: El paciente está enfermo y el modelo predice que el paciente está sano (error grave).

FP: El paciente está sano y el modelo predice que el paciente enfermo (falsa alarma).

Métrica	Fórmula / Significado	Enfoque
Exactitud (Accuracy)	(TP + TN) / Total	Porcentaje de aciertos globales.
Precision	TP / (TP + FP)	Cuantas predicciones positivas fueron correctas. Confiabilidad en las predicciones positivas del modelo.
Exhaustividad (Recall / Sensitivity) **'True Positive Rate'**	TP / (TP + FN)	De todos los positivos reales, Cuantos logró identificar el modelo. No dejar escapar casos críticos.
F1-score	2 × (Precision × Recall)/(Precision + Recall)	Equilibrio entre precisión y recall.
False Positive Rate	FP / (FP + TN)	De todos los negativos reales, ¿cuántos clasificó incorrectamente como positivos?

Accuracy no siempre basta.
Si el 95 % de los casos son "No", un modelo que siempre dice "No" tiene 95 % accuracy… pero **no sirve**.

Matriz de Buen Desempeño (Modelo Preciso y Confiable)

Los números altos están en la diagonal de aciertos (**TP** y **TN**).
Los errores (**FP** y **FN**) son mínimos.

	Predicción: Enfermo (+)	Predicción: Sano (-)
Real: Enfermo (+)	**95 (TP)**	5 (FN)
Real: Sano (-)	3 (FP)	**97 (TN)**

Accuracy = (TP + TN) / Total = (95 + 97) / (95+97+3+5) = 192/200 = 96%

Matriz de Mal Desempeño

El modelo no aprendió ningún patrón y predice al azar (como lanzar una moneda).

	Predicción: Enfermo (+)	Predicción: Sano (-)
Real: Enfermo (+)	**52 (TP)**	48 (FN)
Real: Sano (-)	49 (FP)	**51 (TN)**

Accuracy = (TP + TN) / Total = (52 + 51) / (95+97+3+5) = 103/200 = 51.5%

Matriz de Mal Desempeño

El modelo aprende a responder siempre lo mismo (muy común con datos desbalanceados).

	Predicción: Enfermo (+)	Predicción: Sano (-)
Real: Enfermo (+)	0 (TP)	**100 (FN)**
Real: Sano (-)	0 (FP)	**100 (TN)**

Accuracy = (TP + TN) / Total = (0 + 100) / (95+97+3+5) = 100/200 = 50%

El modelo ignora por completo la clase positiva.

Curva ROC

(*Receiver Operating Characteristic*)
Características operativas del receptor

Supongamos que tenemos un modelo que predice si un paciente tiene una enfermedad:

- `1` = tiene la enfermedad
- `0` = no tiene la enfermedad

Los modelos de clasificación normalmente devuelven la probabilidad de pertenecer a una clase:

Paciente	Probabilidad de enfermedad
A	0.95
B	0.82
C	0.63
D	0.41
E	0.20

Un modelo que predice si un paciente tiene una enfermedad:

- `1` = tiene la enfermedad
- `0` = no tiene la enfermedad

¿A partir de qué probabilidad decimos que el paciente está enfermo?

Threshold = 0.50

- Probabilidad ≥ 0.50 → Enfermo
- Probabilidad < 0.50 → No enfermo

Paciente	Probabilidad de enfermedad
A	0.95
B	0.82
C	0.63
D	0.41
E	0.20

→ Detectaremos 3 enfermos

¿Que pasa si cambiamos el threshold?

Threshold = 0.80

- Probabilidad ≥ 0.80 → **Enfermo**
- Probabilidad < 0.80 → **No enfermo**

Paciente	Probabilidad de enfermedad
A	0.95
B	0.82
C	0.63
D	0.41
E	0.20

Somos más estrictos para decir que alguien está enfermo.

→ Detectaremos menos enfermos (2).

→ Pero también tendremos **menos falsos positivos (falsas alarmas)**.

¿Que pasa si cambiamos el threshold?

Threshold = 0.30

- Probabilidad ≥ 0.30 → **Enfermo**
- Probabilidad < 0.30 → **No enfermo**

Paciente	Probabilidad de enfermedad
A	0.95
B	0.82
C	0.63
D	0.41
E	0.20

Somos más permisivos.

→ Detectaremos más enfermos (4).

→ Pero tendremos **más falsos positivos (falsas alarmas)**.

La curva ROC nos permite visualizar ese balance en subir / bajar el umbral de decisión **y su impacto en TPR y FPR.**

Positivo = Enfermo
Negativo = Sano

TPR — True Positive Rate (Recall/Sensitivity)

De todos los positivos reales, ¿cuántos detectó correctamente mi modelo?

$$TPR = \frac{TP}{TP + FN}$$

Tenemos 100 pacientes enfermos. TP+FN=100

El modelo detecta correctamente 80. TP=80

$$TPR = \frac{80}{100} = 0.80$$

FPR — False Positive Rate

De todos los negativos reales, ¿cuántos clasificó incorrectamente como positivos?

$$FPR = \frac{FP}{FP + TN}$$

Tenemos 100 pacientes sanos. FP+TN=100

El modelo identifica incorrectamente a 20 como enfermos. FP=20

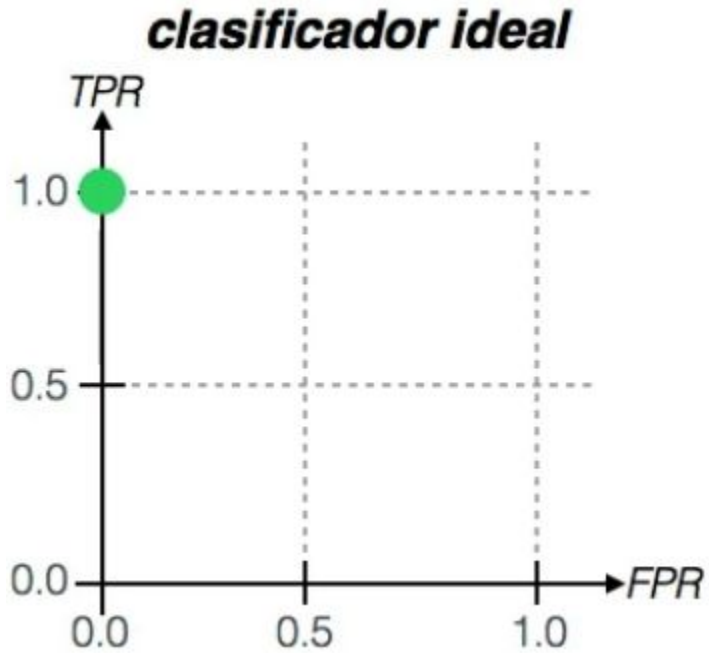
$$FPR = \frac{20}{100} = 0.20$$

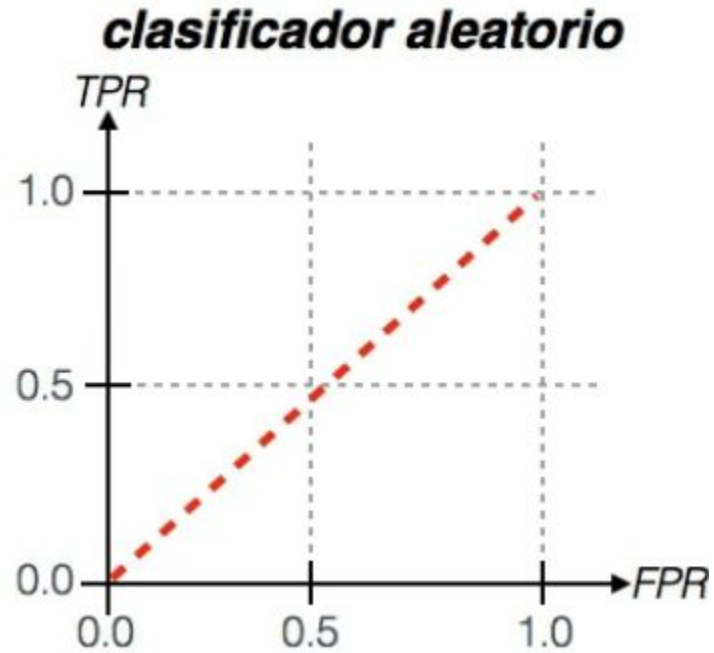
Curva ROC

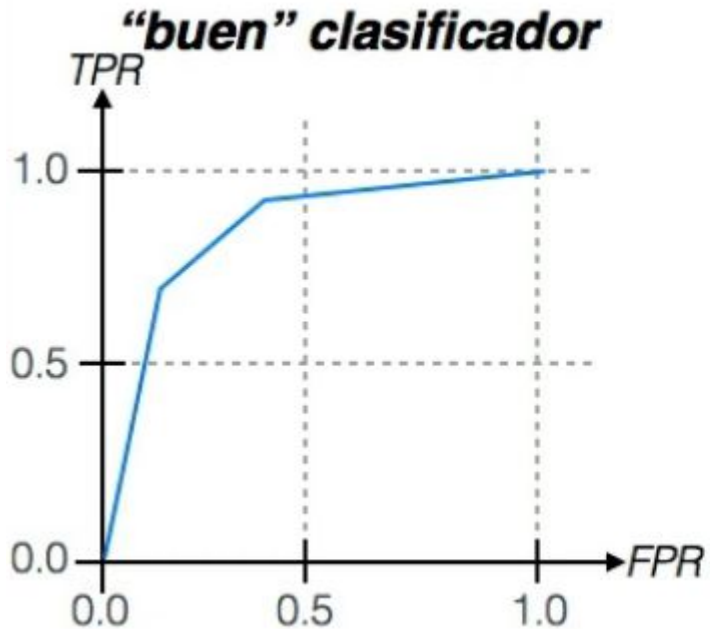
TPR vs. FPR

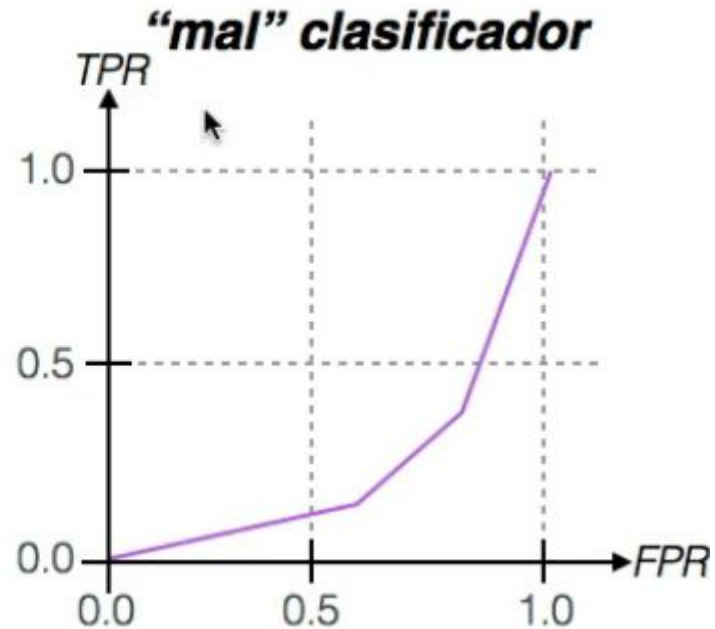
- Eje X → FPR
- Eje Y → TPR

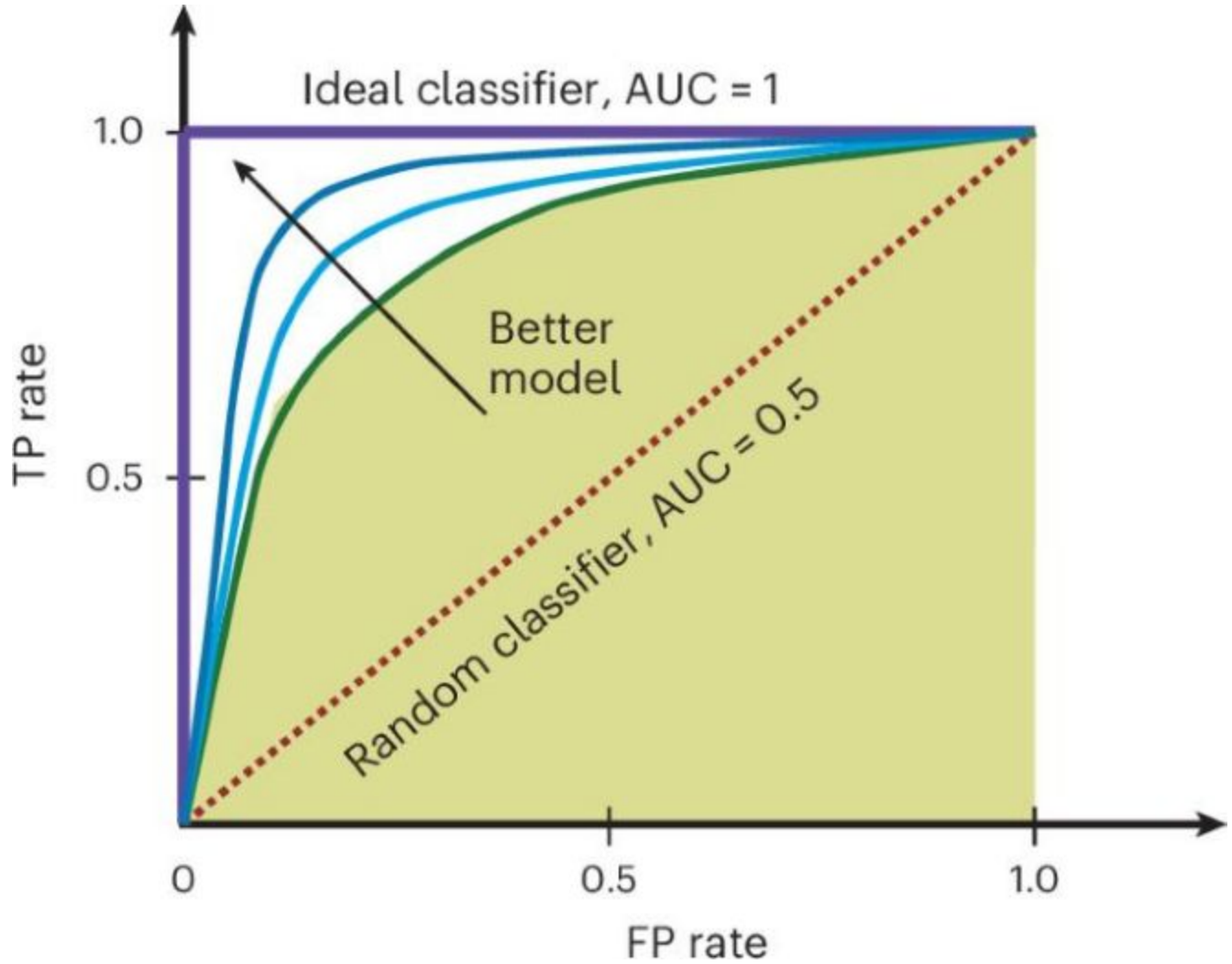
Threshold	TPR	FPR
0.90	0.30	0.02
0.70	0.60	0.10
0.50	0.80	0.20
0.30	0.95	0.45
0.10	1.00	0.90


clasificador ideal
TPR
1.0
0.5
0.0
FPR
0.0
0.5
1.0


clasificador aleatorio
TPR
1.0
0.5
0.0
FPR
0.0
0.5
1.0


“buen” clasificador
TPR
1.0
0.5
0.0
FPR
0.0
0.5
1.0

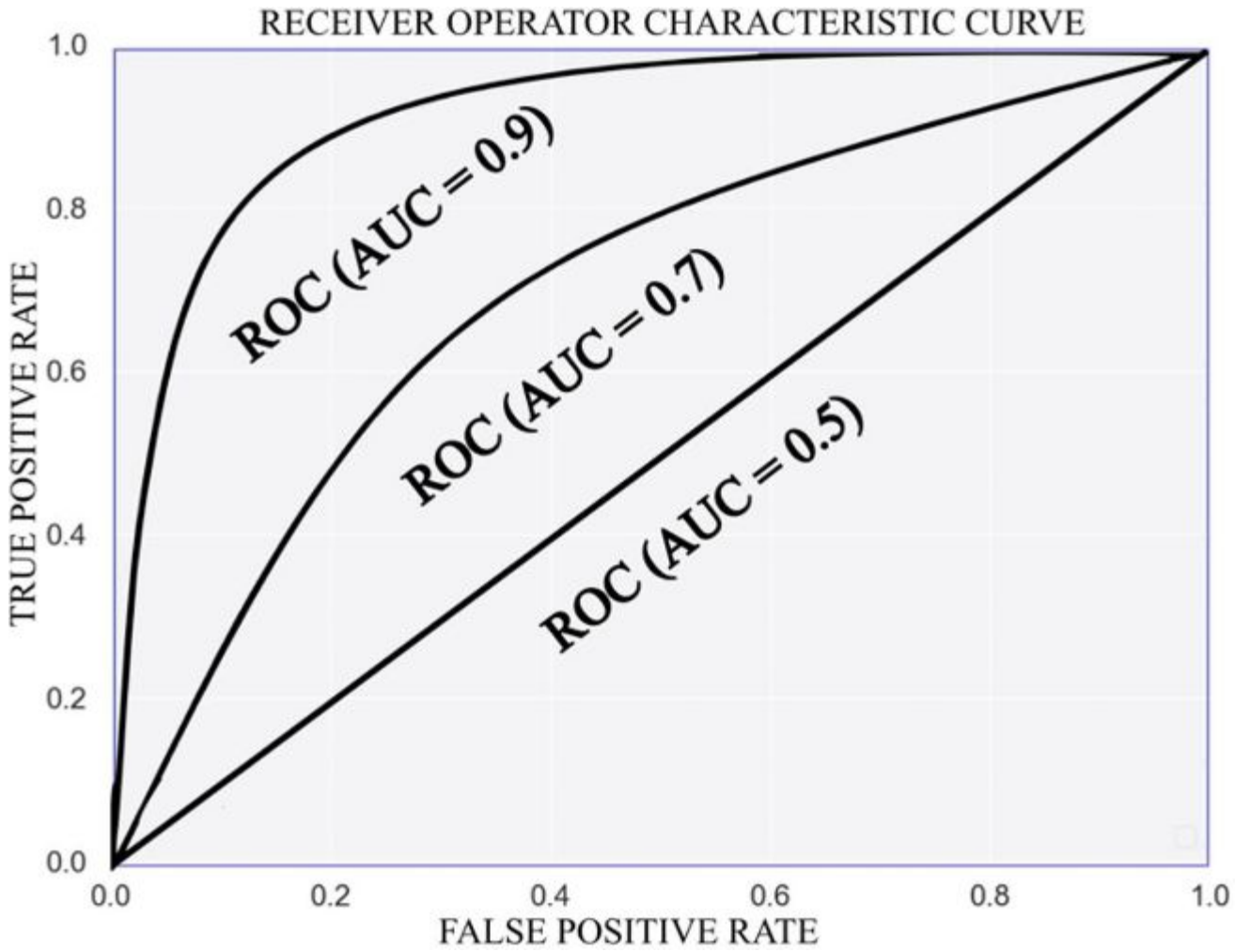

“mal” clasificador
TPR
1.0
0.5
0.0
FPR
0.0
0.5
1.0


Ideal classifier, AUC = 1
1.0
0.5
TP rate
Better
model
Random classifier, AUC = 0.5
0
0.5
1.0
FP rate

AUC
Area Under the Curve

Es el área debajo de la curva ROC.

AUC	Interpretación
0.50	Sin capacidad de discriminación
0.60	Muy débil
0.70	Aceptable
0.80	Bueno
0.90	Muy bueno
1.00	Perfecto



Práctica 4



2. Aplicaciones de ML en Clasificación

2.1 Revisión de modelos y métricas

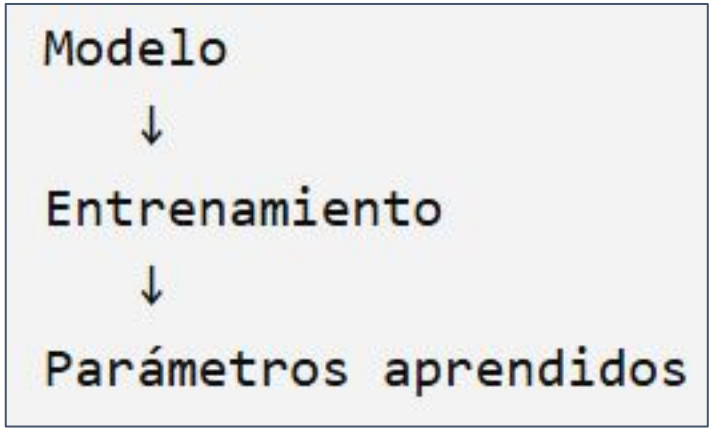
2.2 Randomized Search

2.3 Optimización Bayesiana

Todo modelo de machine learning tiene hiperparámetros: valores que no se aprenden del dato, sino que se fijan antes de entrenar (por ejemplo, la profundidad máxima de un árbol, el número de vecinos en KNN, o el parámetro de regularización C en SVM).

Elegir bien estos valores puede ser la diferencia entre un modelo mediocre y uno excelente, pero probarlos "a mano" es poco sistemático.

Problema del GridSeach



`n_estimators; max_features;`
`min_samples_split; max_depth`

¿Qué combinación de hiperparámetros produce el mejor modelo?

```
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 10]
}
```

Grid Search prueba **todas** las combinaciones:
3*3 =9

n_estimators	max_depth
50	3
50	5
50	10
100	3
100	5
100	10
200	3
200	5
200	10

```
n_estimators = [50, 100, 200, 300, 500]
max_depth = [3, 5, 10, 15, 20]
min_samples_split = [2, 5, 10]
max_features = ['sqrt', 'log2']
```

Número de combinaciones: 5×5×3×2=150

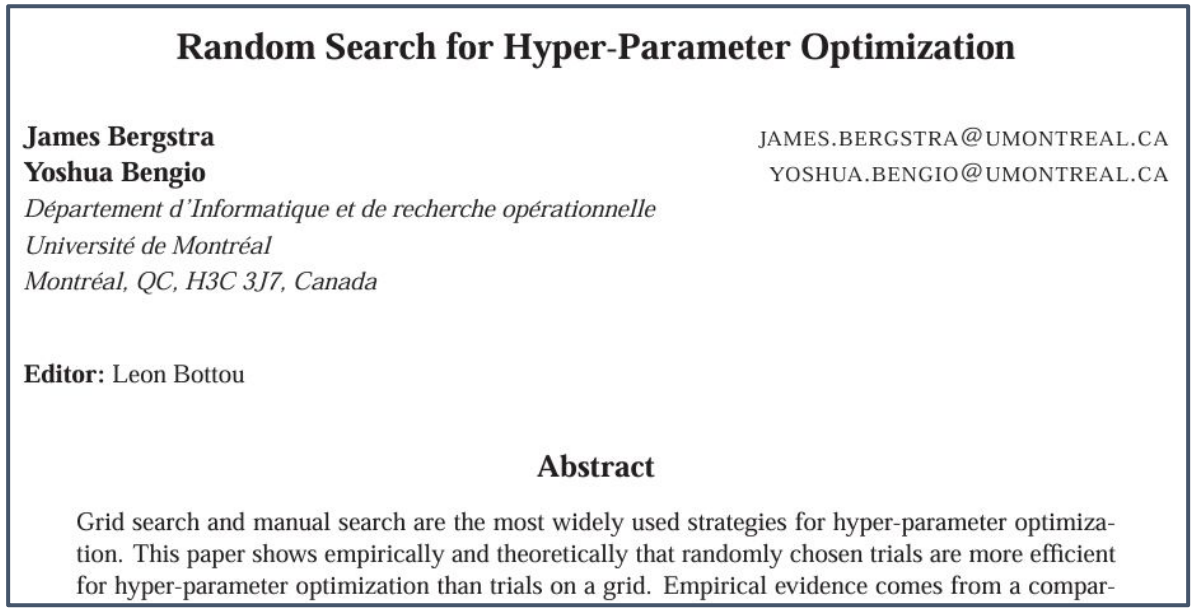
Mas: cv=5

150×5=750 modelos entrenados

RandomizedSearchCV

Randomized Search es una estrategia alternativa.

En lugar de probar **TODAS** las combinaciones, seleccionamos aleatoriamente algunas (n_iter = 20), dentro de un rango.



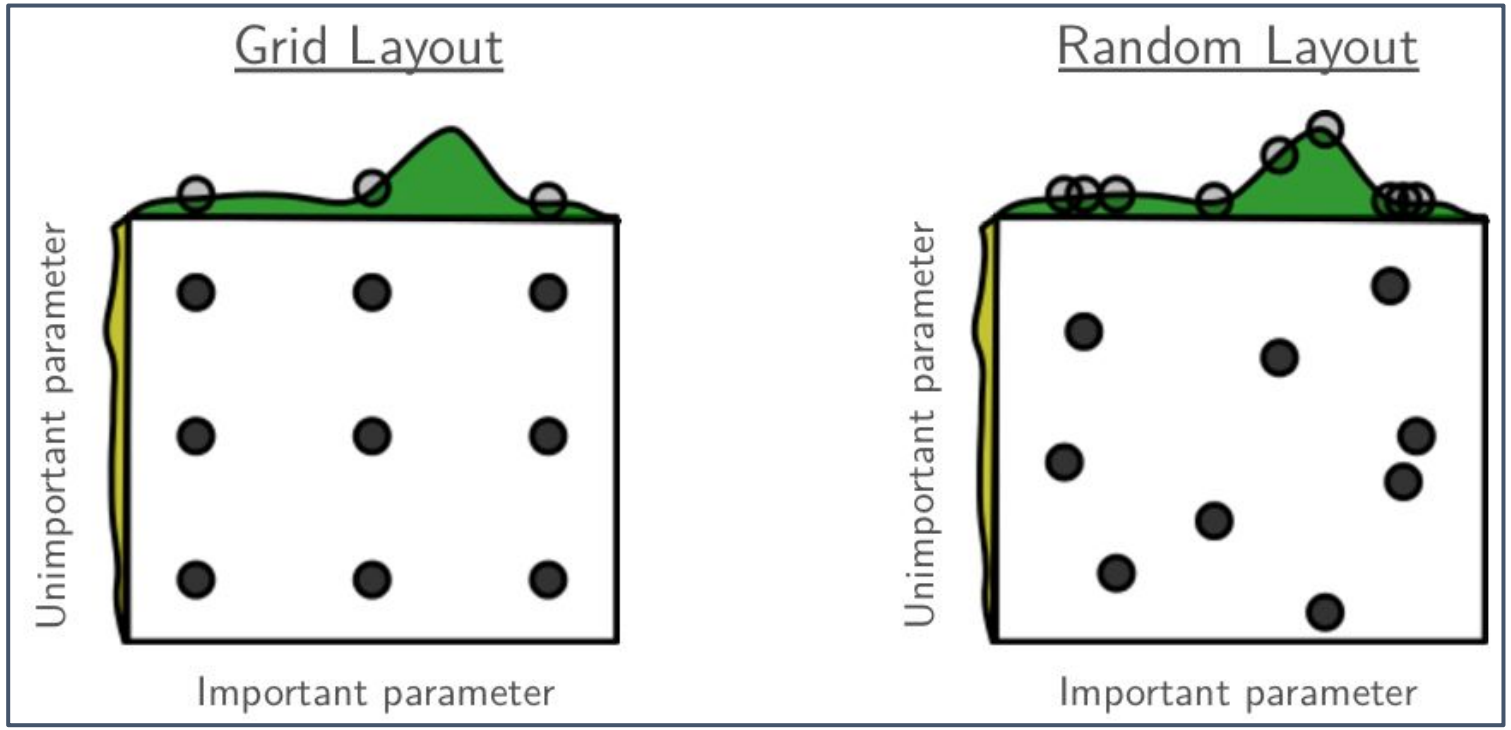
Random Search for Hyper-Parameter Optimization

James Bergstra JAMES.BERGSTRA@UMONTREAL.CA
Yoshua Bengio YOSHUA.BENGIO@UMONTREAL.CA
Département d'Informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC, H3C 3J7, Canada

Editor: Leon Bottou

Abstract

Grid search and manual search are the most widely used strategies for hyper-parameter optimization. This paper shows empirically and theoretically that randomly chosen trials are more efficient for hyper-parameter optimization than trials on a grid. Empirical evidence comes from a compar-



RandomSearch explora un número mayor de valores únicos en las distintas dimensiones dentro del mismo tiempo/presupuesto de cómputo.

Suele encontrar una solución igual o mejor que Grid Search en mucho menos tiempo, aunque no garantiza evaluar una combinación exacta predefinida.

```
Grid Search
150 × 5 = 750 entrenamientos
```

RandomizedSearch
n_iter = 20 * cv=5 = 100 entrenamientos

Grid Search optimiza exhaustivamente;
Randomized Search optimiza eficientemente.

RandomizedSearchCV(model, param_distributions=params, n_iter=20)

n_iter = 20 * cv=5 = 100 entrenamientos

Supongamos que tenemos 4 hiperparámetros:

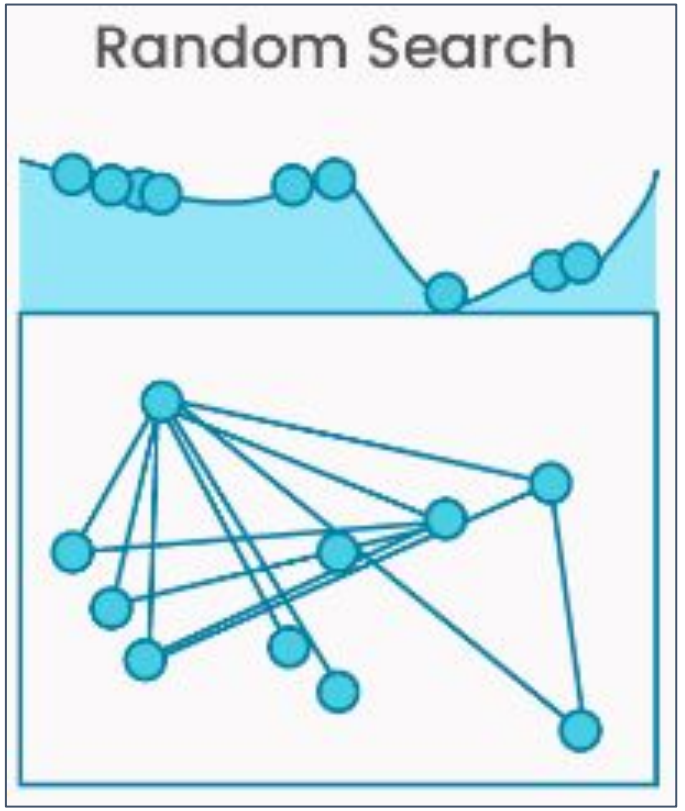
A → importante
B → importante
C → poco importante
D → poco importante

- Grid Search gasta muchos recursos explorando todas las combinaciones.
- Random Search puede encontrar rápidamente buenas regiones del espacio.

RandomSearch puede encontrar rápidamente buenas regiones del espacio.

RS puede explorar más valores distintos de cada hiperparámetro

RS usa una distribución




```
from scipy.stats import randint
param_distributions = { 'max_depth': randint(3, 20)}
```

param_distributions = 3, 4, 5, 6, ..., 19
RS selecciona valores aleatoriamente.

Práctica 5



2. Aplicaciones de ML en Clasificación

2.1 Revisión de modelos y métricas
2.2 Randomized Search
2.3 Optimización Bayesiana

Optimización Bayesiana

A diferencia de *GridSearch* y *RandomSearch*, **utiliza las evaluaciones pasadas** (aprende) para construir un modelo probabilístico de la función objetivo y **decidir estratégicamente dónde evaluar a continuación**.

Cada experimento nos da información sobre dónde vale la pena buscar después.
En lugar de hacer:

Prueba → Prueba → Prueba → Prueba

Hacemos:

Prueba
↓
Aprende de los resultados
↓
Decide dónde probar
↓
Prueba
↓
Aprende
↓
Decide dónde probar
↓
...

Se busca maximizar/minimizar una función f(x) [mse,accuracy,R2,etc].
x es la combinación de los hiperparametros.

En ML avanzado no se tiene una fórmula explícita de f(x) así que no podemos usar la técnica del gradiente.

Dado que no tenemos* una forma analitica de f(x), construiremos un modelo probabilístico que la aproxime a partir de evaluaciones ya hechas… Se suele usar un Proceso Gaussiano.

Componentes de la Optimización Bayesiana

1.- Modelo Sustituto (Surrogate Model)

Esta función, Proceso Gaussiano, intenta aproximar la función de rendimiento.

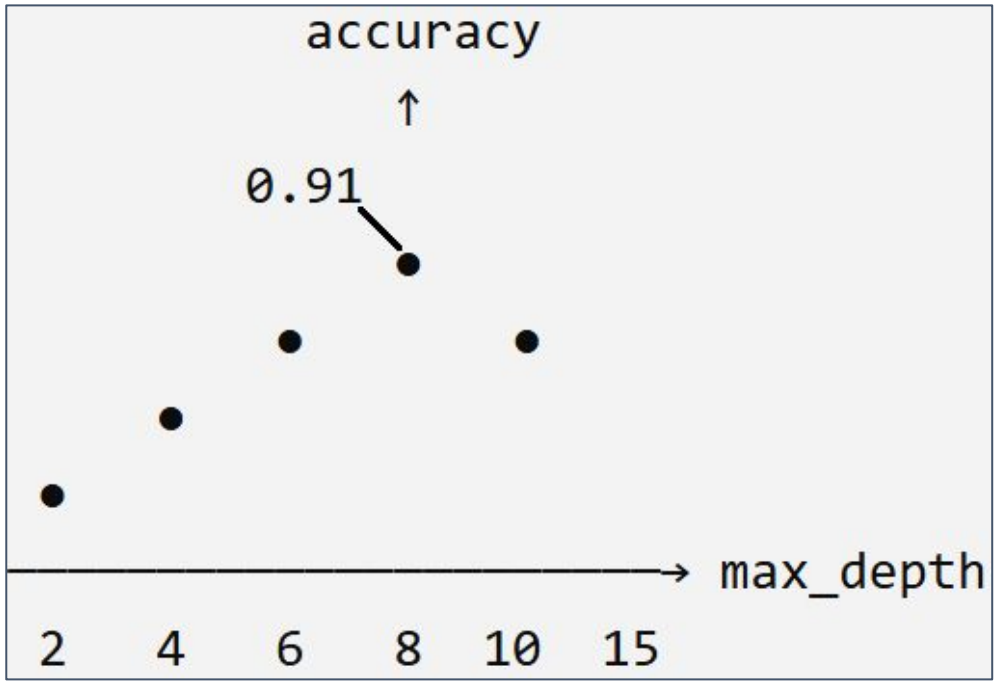
Indica que tan bueno puede ser un punto.

X	F(X)
max_depth	Accuracy real
2	0.72
4	0.80
6	0.87
8	0.91
10	0.90
15	0.88

La función Surrogate aprenderá:

"Probablemente alrededor de max_depth = 8 hay una buena solución."

X	F(X)
max_depth	Accuracy real
2	0.72
4	0.80
6	0.87
8	0.91
10	0.90
15	0.88



El Proceso Gaussiano para cada punto F(x) devuelve además media y desviación estándar/varianza.

2.- Función de Adquisición (Acquisition function)

Tomando como base al modelo sustituto, esta función decide el siguiente punto X a evaluar, y por lo tanto a los próximos hiperparametros a probar.

La FA toma la decisión de sgte. punto balanceando información de 2 fuentes.

Explotación

Buscar cerca de zonas que ya sabemos que son buenas.

Explotar significa evaluar puntos donde la Media es alta

Exploración

Investigar zonas que todavía conocemos poco.

Un punto con varianza alta indica que el modelo conoce poco de ese espacio.

Explorar significa evaluar puntos donde la Varianza es alta

Explotación
Buscar cerca de zonas que ya sabemos que son buenas.
Basado en un valor alto de media.

Exploración
Investigar zonas que todavía conocemos poco.
Basado en un valor alto de varianza.

Alta Media, Baja Varianza: Zona muy conocida con excelente resultado → ***Explotar.***

Baja Media, Alta Varianza: Zona desconocida que podría ocultar un mejor resultado global → ***Explorar.***

Baja Media, Baja Varianza: Zona conocida y con mal resultado → ***Descartar.***

Gracias al aprendizaje, Bayesian Optimization puede alcanzar buenos resultados con **menos trials**, especialmente cuando cada evaluación del modelo es costosa.

Práctica 6

Muchas Gracias